

Importing the data

Installing needed packages

```
## package 'openxlsx' successfully unpacked and MD5 sums checked
```

```
##  
## The downloaded binary packages are in  
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
##  
## There is a binary version available but the source version is later:  
## binary source needs_compilation  
## writexl 1.5.3 1.5.4 TRUE  
##  
## Binaries will be installed  
## package 'writexl' successfully unpacked and MD5 sums checked
```

```
##  
## The downloaded binary packages are in  
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
## package 'tidygeocoder' successfully unpacked and MD5 sums checked  
##  
## The downloaded binary packages are in  
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

Give an overview of dataset.

```
setwd("C:/Users/knigh/Downloads/Submission/Submission")  
collisions = read_xlsx("Motor_Vehicle_Collisions-500K.xlsx")
```

Exploring The Data

Looking at the structure of the data.

```
str(collisions)
```

```
## tibble [472,244 × 29] (S3: tbl_df/tbl/data.frame)
## $ CRASH.DATE           : POSIXct[1:472244], format: "2024-12-30" "2024-12-29" ...
## $ CRASH.TIME           : POSIXct[1:472244], format: "1899-12-31 12:30:00" "1899-12-3
1 06:02:00" ...
## $ BOROUGH              : chr [1:472244] "QUEENS" "MANHATTAN" NA NA ...
## $ ZIP.CODE             : num [1:472244] 11367 10002 NA NA NA ...
## $ LATITUDE             : num [1:472244] 40.7 40.7 40.7 40.6 40.7 ...
## $ LONGITUDE            : num [1:472244] -73.8 -74 -73.9 -74.2 -73.8 ...
## $ LOCATION             : chr [1:472244] "(40.73002, -73.82347)" "(40.719864, -73.991
7)" "(40.729515, -73.87149)" "(40.605522, -74.18429)" ...
## $ ON.STREET.NAME       : chr [1:472244] "JEWEL AVE" NA "WOODHAVEN BLVD" "WEST SHORE
EXPRESSWAY" ...
## $ CROSS.STREET.NAME    : chr [1:472244] "MAIN ST" NA "ELIOT AVE" NA ...
## $ OFF.STREET.NAME      : chr [1:472244] NA "38          DELANCEY ST" NA NA ...
## $ NUMBER.OF.PERSONS.INJURED : num [1:472244] 1 0 0 3 0 2 0 0 0 0 ...
## $ NUMBER.OF.PERSONS.KILLED : num [1:472244] 0 0 0 0 0 0 0 0 0 0 ...
## $ NUMBER.OF.PEDESTRIANS.INJURED: num [1:472244] 0 0 0 0 0 0 0 0 0 0 ...
## $ NUMBER.OF.PEDESTRIANS.KILLED : num [1:472244] 0 0 0 0 0 0 0 0 0 0 ...
## $ NUMBER.OF.CYCLIST.INJURED : num [1:472244] 1 0 0 0 0 0 0 0 0 0 ...
## $ NUMBER.OF.CYCLIST.KILLED : num [1:472244] 0 0 0 0 0 0 0 0 0 0 ...
## $ NUMBER.OF.MOTORIST.INJURED : num [1:472244] 0 0 0 3 0 2 0 0 0 0 ...
## $ NUMBER.OF.MOTORIST.KILLED : num [1:472244] 0 0 0 0 0 0 0 0 0 0 ...
## $ CONTRIBUTING.FACTOR.VEHICLE.1: chr [1:472244] "Unspecified" "Unspecified" "Turning Imprope
rly" "Driver Inattention/Distractio" ...
## $ CONTRIBUTING.FACTOR.VEHICLE.2: chr [1:472244] NA NA "Unspecified" "Unspecified" ...
## $ CONTRIBUTING.FACTOR.VEHICLE.3: chr [1:472244] NA NA NA NA ...
## $ CONTRIBUTING.FACTOR.VEHICLE.4: chr [1:472244] NA NA NA NA ...
## $ CONTRIBUTING.FACTOR.VEHICLE.5: chr [1:472244] NA NA NA NA ...
## $ COLLISION_ID         : num [1:472244] 4782671 4782455 4782577 4782966 4781980 ...
## $ VEHICLE.TYPE.CODE.1   : chr [1:472244] "Bike" "Station Wagon/Sport Utility Vehicle"
"Station Wagon/Sport Utility Vehicle" "Station Wagon/Sport Utility Vehicle" ...
## $ VEHICLE.TYPE.CODE.2   : chr [1:472244] NA "Sedan" "Station Wagon/Sport Utility Vehi
cle" "Station Wagon/Sport Utility Vehicle" ...
## $ VEHICLE.TYPE.CODE.3   : chr [1:472244] NA NA NA NA ...
## $ VEHICLE.TYPE.CODE.4   : chr [1:472244] NA NA NA NA ...
## $ VEHICLE.TYPE.CODE.5   : chr [1:472244] NA NA NA NA ...
```

29 variables in Total- 2 POSIXct date features, 15 character features, 12 numeric features.

Taking a closer look at the **POSIX variables**

```
summary(collisions[c("CRASH.DATE", "CRASH.TIME")])
```

##	CRASH.DATE	CRASH.TIME
##	Min. :2020-01-01 00:00:00.00	Min. :1899-12-31 00:00:00.00
##	1st Qu.:2021-01-22 00:00:00.00	1st Qu.:1899-12-31 08:55:00.00
##	Median :2022-02-18 00:00:00.00	Median :1899-12-31 14:00:00.00
##	Mean :2022-03-04 03:45:38.75	Mean :1899-12-31 13:16:14.65
##	3rd Qu.:2023-04-18 00:00:00.00	3rd Qu.:1899-12-31 18:04:00.00
##	Max. :2024-12-30 00:00:00.00	Max. :1899-12-31 23:59:00.00

Can see range of time is from 1/1/2020 - 12/30/2024. The time range is full 24 hours. The 1899-12-31 component is a common place holder for when time is stored separately from the date, using 00:00:00 on 1899-12-31 as a default date.

Checking if these columns have any NAs. If no NA's we will likely combine the two columns together to form a CRASH.DATE_TIME feature. **Transformation steps will occur after reviewing all columns to ensure big picture is understood before altering the data.**

```
colSums(is.na(collisions[, c("CRASH.DATE", "CRASH.TIME"))))
```

```
## CRASH.DATE CRASH.TIME  
##           0           0
```

No NAs are found in these columns. GOOD NEWS!

Looking at the distribution of time through the years

```
sum(format(collisions$CRASH.DATE, "%Y") == "2024")
```

```
## [1] 48318
```

```
sum(format(collisions$CRASH.DATE, "%Y") == "2023")
```

```
## [1] 96575
```

```
sum(format(collisions$CRASH.DATE, "%Y") == "2022")
```

```
## [1] 103884
```

```
sum(format(collisions$CRASH.DATE, "%Y") == "2021")
```

```
## [1] 110551
```

Note that there is not many records from 2024 even though the range of the of the data covers up to 12/30/24.

Now looking at the summaries of our **numeric columns**, starting with the **location features**.

```
summary(collisions[c("ZIP.CODE", "LATITUDE", "LONGITUDE")])
```

```
##      ZIP.CODE      LATITUDE      LONGITUDE
## Min.   :10000    Min.   : 0.00    Min.   : -74.25
## 1st Qu.:10457    1st Qu.:40.67    1st Qu.: -73.96
## Median :11208    Median :40.71    Median : -73.92
## Mean   :10895    Mean   :40.41    Mean   : -73.35
## 3rd Qu.:11238    3rd Qu.:40.78    3rd Qu.: -73.87
## Max.   :11697    Max.   :40.91    Max.   :  0.00
## NA's   :158753   NA's   :37031    NA's   :37031
```

Since Zipcode, Latitude, and Longitude represent locations, summary statistics like mean, min, max, and quartiles are not particularly useful for interpretation. Additionally, the 0 values in Latitude and Longitude likely indicate missing data, as our dataset focuses on New York, where valid coordinates should not be (0,0). The most meaningful insights from summarizing these features include the count of missing values (NAs) and the median Zipcode, which may indicate where a significant number of crashes occur. However, a large number of missing values in these columns could impact the reliability of location-based analysis

Looking for 0's in Lat and Long as they are probably representing missing values as well.

```
colSums(collisions[, c("LATITUDE", "LONGITUDE")] == 0, na.rm = TRUE)
```

```
## LATITUDE LONGITUDE
##      3326      3326
```

These 0 values stack on top of the NA count for the columns above.

Additionally zipcode 10000 shown as the MIN is not a zipcode in New York. However 10001 is. Weird I know. I looked at these manually by ascending the data on table window, thankfully there is not many of these and they all align with Manahattan. Thank goodness. For any other inconsistent zipcodes, we may need to do some research.

Now looking into Number of **people injured**.

```
summary(collisions[c("NUMBER.OF.PERSONS.INJURED", "NUMBER.OF.PEDESTRIANS.INJURED", "NUMBER.OF.CYC
LIST.INJURED", "NUMBER.OF.MOTORIST.INJURED")])
```

```
## NUMBER.OF.PERSONS.INJURED NUMBER.OF.PEDESTRIANS.INJURED
## Min.   : 0.0000      Min.   : 0.00000
## 1st Qu.: 0.0000      1st Qu.: 0.00000
## Median : 0.0000      Median : 0.00000
## Mean   : 0.4882      Mean   : 0.07847
## 3rd Qu.: 1.0000      3rd Qu.: 0.00000
## Max.   :40.0000      Max.   :19.00000
## NA's   :1
## NUMBER.OF.CYCLIST.INJURED NUMBER.OF.MOTORIST.INJURED
## Min.   :0.00000      Min.   : 0.0000
## 1st Qu.:0.00000      1st Qu.: 0.0000
## Median :0.00000      Median : 0.0000
## Mean   :0.04899      Mean   : 0.3441
## 3rd Qu.:0.00000      3rd Qu.: 0.0000
## Max.   :3.00000      Max.   :40.0000
##
```

There is only 1 NA in all these columns, that is great news! We'll just remove this record in the cleaning stage. Note that most of the statistics are 0, meaning most collisions did not have any injuries. When predicting crash severity this will probably create some class imbalance.

From the website we know that Number of Persons injured should match the combined total of pedestrians, cyclist, and motorist. We want to confirm this by adding up the three columns (pedestrians, cyclist, and motorist) and seeing where it does not align with Number of Persons.

```
mismatch_indexes = which((collisions$NUMBER.OF.PEDESTRIANS.INJURED +
                           collisions$NUMBER.OF.CYCLIST.INJURED +
                           collisions$NUMBER.OF.MOTORIST.INJURED) != collisions$NUMBER.OF.PERSON
                           S.INJURED)

mismatch_count = length(mismatch_indexes)

mismatch_count
```

```
## [1] 7496
```

We are seeing 7496 records not align with the Number of Persons. This is a relatively small amount in our data of 500k records, 1.6%, so we may just remove these when cleaning.

Now looking at number of **people killed**

```
summary(collisions[c("NUMBER.OF.PERSONS.KILLED", "NUMBER.OF.PEDESTRIANS.KILLED", "NUMBER.OF.CYCLIST.KILLED", "NUMBER.OF.MOTORIST.KILLED")])
```

```
##  NUMBER.OF.PERSONS.KILLED  NUMBER.OF.PEDESTRIANS.KILLED  NUMBER.OF.CYCLIST.KILLED
##  Min.      :0.000000      Min.      :0.000000      Min.      :0.000000
##  1st Qu.:0.000000      1st Qu.:0.000000      1st Qu.:0.000000
##  Median :0.000000      Median :0.000000      Median :0.000000
##  Mean   :0.002674      Mean   :0.001141      Mean   :0.0002245
##  3rd Qu.:0.000000      3rd Qu.:0.000000      3rd Qu.:0.000000
##  Max.   :5.000000      Max.   :4.000000      Max.   :1.000000
##  NUMBER.OF.MOTORIST.KILLED
##  Min.      :0.000000
##  1st Qu.:0.000000
##  Median :0.000000
##  Mean   :0.001205
##  3rd Qu.:0.000000
##  Max.   :5.000000
```

Again many more 0's and very low averages meaning not many people were killed in the crashes recorded. Signifying further class imbalance for our classifier models. No NA's here!!

The pedestrians, cyclists, and motorist should also align with number of persons killed. Checking again to see consistency in the totals.

```

mismatch_indexes2 = which((collisions$NUMBER.OF.PEDESTRIANS.KILLED +
                           collisions$NUMBER.OF.CYCLIST.KILLED +
                           collisions$NUMBER.OF.MOTORIST.KILLED) != collisions$NUMBER.OF.PERSON
                           S.KILLED)

mismatch_count2 = length(mismatch_indexes2)

mismatch_count2

```

```
## [1] 49
```

Thankfully we only have 49 misalignment here. We can definitely remove these in the cleaning stage.

For the numeric column **Collision ID**. There is no need to check the summary as this is a unique identifier, giving little information for overall statistics. The main thing is that all collision IDs should be distinct.

```
n_distinct(collisions$COLLISION_ID)
```

```
## [1] 472244
```

There are 472244 unique IDs, and there are 472244 records in our data. They are all distinct! :)

Now to review the **character features**, we will start by looking at the **location columns**

Looking at the number of NA's per column

```

collisions %>%
  summarise(across(c(BOROUGH, LOCATION, ON.STREET.NAME, CROSS.STREET.NAME, OFF.STREET.NAME), ~ sum(
    is.na(.)))) %>%
  t()

```

```

##           [,1]
## BOROUGH    158696
## LOCATION   37031
## ON.STREET.NAME 127914
## CROSS.STREET.NAME 248778
## OFF.STREET.NAME 344334

```

We have ALOT of NA's in these columns.

Now looking at the distinct values

```

collisions %>%
  summarise(across(c(BOROUGH, LOCATION, ON.STREET.NAME, CROSS.STREET.NAME, OFF.STREET.NAME), ~ n_
    _distinct(.x, na.rm = TRUE))) %>%
  t()

```

```
##           [,1]
## BOROUGH      5
## LOCATION    133575
## ON.STREET.NAME 7423
## CROSS.STREET.NAME 8431
## OFF.STREET.NAME 91249
```

Above is all the distinct values in the columns, with NA's removed. Borough shows the number 5, which is what we would expect as there are 5 boroughs in NYC. These other features have a lot of unique values, which is also to be expected as they define the specific location of crashes.

Reviewing the mode

```
#Borough
temp = table(collisions$BOROUGH)
temp[which.max(table(collisions$BOROUGH))]
```

```
## BROOKLYN
##      108570
```

```
#Location
temp = table(collisions$LOCATION)
temp[which.max(table(collisions$LOCATION))]
```

```
## (0, 0)
##      3326
```

```
#On Street
temp = table(collisions$ON.STREET.NAME)
temp[which.max(table(collisions$ON.STREET.NAME))]
```

```
## BELT PARKWAY
##           7306
```

```
#Cross Street
temp = table(collisions$CROSS.STREET.NAME)
temp[which.max(table(collisions$CROSS.STREET.NAME))]
```

```
## 3 AVENUE
##      2194
```

```
#Off Street
temp = table(collisions$OFF.STREET.NAME)
temp[which.max(table(collisions$OFF.STREET.NAME))]
```

```
## 772      EDGEWATER ROAD
##              110
```

Here we can see the most common crash borough is Brooklyn(However this is before replacing NA's). The column Location, is expected to have many unique combinations as it is coordinates, (0,0) is basically an NA, so it is difficult to find the mode for this one. The last 3 show streets with the most common accidents.

Next reviewing the **Contributing factors**. There are multiple contributing factor columns that show if a crash had multiple reasons for occurring.

Starting with the NA's.

```
collisions %>%
  summarise(across(c(CONTRIBUTING.FACTOR.VEHICLE.1, CONTRIBUTING.FACTOR.VEHICLE.2, CONTRIBUTING.
    FACTOR.VEHICLE.3, CONTRIBUTING.FACTOR.VEHICLE.4, CONTRIBUTING.FACTOR.VEHICLE.5), ~ sum(is.na
    (.)))) %>%
  t()
```

```
##              [,1]
## CONTRIBUTING.FACTOR.VEHICLE.1  2632
## CONTRIBUTING.FACTOR.VEHICLE.2 107730
## CONTRIBUTING.FACTOR.VEHICLE.3 426928
## CONTRIBUTING.FACTOR.VEHICLE.4 460047
## CONTRIBUTING.FACTOR.VEHICLE.5 468594
```

The NA's increase throughout factors 1-5. Showing that almost all crashes have at least 1 factor, about 77% of crashes had two factors, and less than 1% have 3 factors or more.

How many factors are distinct?

```
collisions %>%
  summarise(across(c(CONTRIBUTING.FACTOR.VEHICLE.1, CONTRIBUTING.FACTOR.VEHICLE.2, CONTRIBUTING.
    FACTOR.VEHICLE.3, CONTRIBUTING.FACTOR.VEHICLE.4, CONTRIBUTING.FACTOR.VEHICLE.5), ~ n_distinct(
    x, na.rm = TRUE))) %>%
  t()
```

```
##              [,1]
## CONTRIBUTING.FACTOR.VEHICLE.1   55
## CONTRIBUTING.FACTOR.VEHICLE.2   53
## CONTRIBUTING.FACTOR.VEHICLE.3   38
## CONTRIBUTING.FACTOR.VEHICLE.4   25
## CONTRIBUTING.FACTOR.VEHICLE.5   15
```

There appears to be a lot of repetition in the factors, which is good! we will have to check if they are the same across all columns. Since they are a low number, we may be able to do this manually.

Now reviewing the mode.


```
#Factor 1
temp = table(collisions$CONTRIBUTING.FACTOR.VEHICLE.1)
temp[which.max(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.1))]
```

```
## Unspecified
##      117029
```

```
#Factor 2
temp = table(collisions$CONTRIBUTING.FACTOR.VEHICLE.2)
temp[which.max(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.2))]
```

```
## Unspecified
##      309314
```

```
#Factor 3
temp = table(collisions$CONTRIBUTING.FACTOR.VEHICLE.3)
temp[which.max(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.3))]
```

```
## Unspecified
##      42468
```

```
#Factor 4
temp = table(collisions$CONTRIBUTING.FACTOR.VEHICLE.4)
temp[which.max(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.4))]
```

```
## Unspecified
##      11525
```

```
#Factor 5
temp = table(collisions$CONTRIBUTING.FACTOR.VEHICLE.5)
temp[which.max(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.5))]
```

```
## Unspecified
##      3443
```

Well this is not great. Looks like we have alot of unspecified reasons. Seeing as most of the Factor 2, 3, 4 and 5 is unspecified, this builds a good case to remove them. Doing the math of subtracting the # of NA's from the total rows, I see we have 364,514 Factor 2 columns that are not NA, out of that 309314 is unspecified..thats 85%. It just gets worse for Factors 3, 4, and 5. So it appears these extra columns do not offer much information.

Looking at what the top 10 factors are for CONTRIBUTING.FACTOR.VEHICLE.1, the most informative column.

```
head(sort(table(collisions$CONTRIBUTING.FACTOR.VEHICLE.1), decreasing = TRUE), 10)
```

```
##
##           Unspecified Driver Inattention/Distracted
##           117029                               116839
## Failure to Yield Right-of-Way           Following Too Closely
##           31973                               30732
## Passing or Lane Usage Improper           Passing Too Closely
##           20156                               17039
##           Unsafe Speed                     Backing Unsafely
##           16426                               15127
##           Other Vehicular           Traffic Control Disregarded
##           13540                               12785
```

Looking at **Vehicle Type**. Like contributing factor, vehicle type has multiple columns for all cars involved.

Exploring NA's.

```
collisions %>%
  summarise(across(c(VEHICLE.TYPE.CODE.1,VEHICLE.TYPE.CODE.2,VEHICLE.TYPE.CODE.3, VEHICLE.TYPE.C
ODE.4, VEHICLE.TYPE.CODE.5), ~ sum(is.na(.)))) %>%
  t()
```

```
##           [,1]
## VEHICLE.TYPE.CODE.1  6148
## VEHICLE.TYPE.CODE.2 154883
## VEHICLE.TYPE.CODE.3 430010
## VEHICLE.TYPE.CODE.4 460684
## VEHICLE.TYPE.CODE.5 468734
```

Similar to the contributing factors the NA's increase with additional vehicles. Showing most crashes have a listed vehicle, 67% of crashes have 2 vehicles listed, and less than 1% has 3 or more vehicles.

Let's see how many distinct values there are.

```
collisions %>%
  summarise(across(c(VEHICLE.TYPE.CODE.1,VEHICLE.TYPE.CODE.2,VEHICLE.TYPE.CODE.3, VEHICLE.TYPE.C
ODE.4, VEHICLE.TYPE.CODE.5), ~ n_distinct(.x, na.rm = TRUE))) %>%
  t()
```

```
##           [,1]
## VEHICLE.TYPE.CODE.1  969
## VEHICLE.TYPE.CODE.2 1077
## VEHICLE.TYPE.CODE.3  173
## VEHICLE.TYPE.CODE.4   74
## VEHICLE.TYPE.CODE.5   42
```

There are MANY distinct vehicles listed. Probably meaning either the vehicles are very specific or poor recording on NYC's end.

Below is the mode of these columns

```
#Vehicle Type 1
temp = table(collisions$VEHICLE.TYPE.CODE.1)
temp[which.max(table(collisions$VEHICLE.TYPE.CODE.1))]
```

```
## Sedan
## 219814
```

```
#Vehicle Type 2
temp = table(collisions$VEHICLE.TYPE.CODE.2)
temp[which.max(table(collisions$VEHICLE.TYPE.CODE.2))]
```

```
## Sedan
## 132910
```

```
#Vehicle Type 3
temp = table(collisions$VEHICLE.TYPE.CODE.3)
temp[which.max(table(collisions$VEHICLE.TYPE.CODE.3))]
```

```
## Sedan
## 20715
```

```
#Vehicle Type 4
temp = table(collisions$VEHICLE.TYPE.CODE.4)
temp[which.max(table(collisions$VEHICLE.TYPE.CODE.4))]
```

```
## Sedan
## 5722
```

```
#Vehicle Type 5
temp = table(collisions$VEHICLE.TYPE.CODE.5)
temp[which.max(table(collisions$VEHICLE.TYPE.CODE.5))]
```

```
## Sedan
## 1741
```

The most common crashes appear to be Sedan cars.

Processing Data

Now that we have gotten a good look at our columns lets start to address some of the issues we uncovered.

Handling NA's

Building an NA table for all features

```
#building the empty table
num_of_records = nrow(collisions) # Will used later in calculating the % of missing values
num_of_features = ncol(collisions) #will use to create an empty table
data_summary = as.data.frame(matrix(nrow=num_of_features,ncol = 3))
#num_of_features is used for nrow so that each row in our created data frame aligns with an attribute from our dataset.

#Adding column names
colnames(data_summary) = c("Features", "Missing Values", "% of Missing Values")

#Adding Features
data_summary[,1] = colnames(collisions)

#Calculating number of NA's and There percentages
for(i in 1:num_of_features){
  data_summary[i,2] = sum(is.na(collisions[,i])) #counting the number of missing values in each cust_info attribute
  data_summary[i,3] = round(100*(sum(is.na(collisions[,i])/num_of_records)),2) #turning the count into a percentage
}

print(data_summary)
```

##	Features	Missing Values	% of Missing Values
## 1	CRASH.DATE	0	0.00
## 2	CRASH.TIME	0	0.00
## 3	BOROUGH	158696	33.60
## 4	ZIP.CODE	158753	33.62
## 5	LATITUDE	37031	7.84
## 6	LONGITUDE	37031	7.84
## 7	LOCATION	37031	7.84
## 8	ON.STREET.NAME	127914	27.09
## 9	CROSS.STREET.NAME	248778	52.68
## 10	OFF.STREET.NAME	344334	72.91
## 11	NUMBER.OF.PERSONS.INJURED	1	0.00
## 12	NUMBER.OF.PERSONS.KILLED	0	0.00
## 13	NUMBER.OF.PEDESTRIANS.INJURED	0	0.00
## 14	NUMBER.OF.PEDESTRIANS.KILLED	0	0.00
## 15	NUMBER.OF.CYCLIST.INJURED	0	0.00
## 16	NUMBER.OF.CYCLIST.KILLED	0	0.00
## 17	NUMBER.OF.MOTORIST.INJURED	0	0.00
## 18	NUMBER.OF.MOTORIST.KILLED	0	0.00
## 19	CONTRIBUTING.FACTOR.VEHICLE.1	2632	0.56
## 20	CONTRIBUTING.FACTOR.VEHICLE.2	107730	22.81
## 21	CONTRIBUTING.FACTOR.VEHICLE.3	426928	90.40
## 22	CONTRIBUTING.FACTOR.VEHICLE.4	460047	97.42
## 23	CONTRIBUTING.FACTOR.VEHICLE.5	468594	99.23
## 24	COLLISION_ID	0	0.00
## 25	VEHICLE.TYPE.CODE.1	6148	1.30
## 26	VEHICLE.TYPE.CODE.2	154883	32.80
## 27	VEHICLE.TYPE.CODE.3	430010	91.06
## 28	VEHICLE.TYPE.CODE.4	460684	97.55
## 29	VEHICLE.TYPE.CODE.5	468734	99.26

Vehicle Type

We remove vehicle type code 3, 4, and 5 features because they don't provide much information.

```
# Vehicle Type features: remove columns with high % of missing values
collisions_clean <- collisions %>% select(-VEHICLE.TYPE.CODE.3, -VEHICLE.TYPE.CODE.4, -VEHICLE.T
YPE.CODE.5)
```

removing NA's in type 1 and setting the remaining NA's in type 2 to 'none'

```
# Vehicle Type Code 1: remove NA's
collisions_clean <- collisions_clean[!is.na(collisions_clean$VEHICLE.TYPE.CODE.1),]
# Replacing NA's in vehicle type code 2 with "none"
collisions_clean <- collisions_clean %>%
  mutate(VEHICLE.TYPE.CODE.2 = replace(VEHICLE.TYPE.CODE.2,
                                       is.na(VEHICLE.TYPE.CODE.2),
                                       "none"))
```

Contributing Factor

We remove contributing factor vehicle 3, 4, and 5 features because they don't provide much information.

```
# Contributing Factor features: remove columns with high % of missing values
collisions_clean <- collisions_clean %>% select(-CONTRIBUTING.FACTOR.VEHICLE.3, -CONTRIBUTING.FACTOR.VEHICLE.4, -CONTRIBUTING.FACTOR.VEHICLE.5)
```

removing NA's in factor 1 and setting the remaining NA's in factor 2 to 'none'

```
# Contributing Factor Vehicle 1: remove NA's
collisions_clean <- collisions_clean[!is.na(collisions_clean$CONTRIBUTING.FACTOR.VEHICLE.1), ]
# Replacing NA's in contributing factor vehicle 2 with "none"
collisions_clean <- collisions_clean %>%
  mutate(CONTRIBUTING.FACTOR.VEHICLE.2 = replace(CONTRIBUTING.FACTOR.VEHICLE.2,
                                                  is.na(CONTRIBUTING.FACTOR.VEHICLE.2),
                                                  "none"))
```

Injured/Fatalities

We remove injured/killed pedestrian, cyclists, and motorist columns because it provides redundant information and we want to focus our analysis on the total persons injured/killed.

```
# number of injured/killed features: remove columns pedestrian, cyclists, motorist columns
collisions_clean <- collisions_clean %>% select(-NUMBER.OF.PEDESTRIANS.INJURED, -NUMBER.OF.PEDESTRIANS.KILLED, -NUMBER.OF.CYCLIST.INJURED, -NUMBER.OF.CYCLIST.KILLED, -NUMBER.OF.MOTORIST.INJURED, -NUMBER.OF.MOTORIST.KILLED)
```

Removing the 1 NA in persons injured column.

```
# remove the 1 NA from number of persons injured
collisions_clean <- collisions_clean[!is.na(collisions_clean$NUMBER.OF.PERSONS.INJURED), ]
```

Looking to confirm NA's have been removed

```

#building the empty table
num_of_records = nrow(collisions_clean) # Will used later in calculating the % of missing values
num_of_features = ncol(collisions_clean) #will use to create an empty table
data_summary = as.data.frame(matrix(nrow=num_of_features,ncol = 3))
#num_of_features is used for nrow so that each row in our created data frame aligns with an attribute from our dataset.

#Adding column names
colnames(data_summary) = c("Features", "Missing Values", "% of Missing Values")

#Adding Features
data_summary[,1] = colnames(collisions_clean)

#Calculating number of NA's and There percentages
for(i in 1:num_of_features){
  data_summary[i,2] = sum(is.na(collisions_clean[,i])) #counting the number of missing values in each cust_info attribute
  data_summary[i,3] = round(100*(sum(is.na(collisions_clean[,i])/num_of_records),2) #turning the count into a percentage
}

print(data_summary)

```

##	Features	Missing Values	% of Missing Values
## 1	CRASH.DATE	0	0.00
## 2	CRASH.TIME	0	0.00
## 3	BOROUGH	157201	33.77
## 4	ZIP.CODE	157258	33.78
## 5	LATITUDE	36712	7.89
## 6	LONGITUDE	36712	7.89
## 7	LOCATION	36712	7.89
## 8	ON.STREET.NAME	126088	27.08
## 9	CROSS.STREET.NAME	245962	52.83
## 10	OFF.STREET.NAME	339459	72.92
## 11	NUMBER.OF.PERSONS.INJURED	0	0.00
## 12	NUMBER.OF.PERSONS.KILLED	0	0.00
## 13	CONTRIBUTING.FACTOR.VEHICLE.1	0	0.00
## 14	CONTRIBUTING.FACTOR.VEHICLE.2	0	0.00
## 15	COLLISION_ID	0	0.00
## 16	VEHICLE.TYPE.CODE.1	0	0.00
## 17	VEHICLE.TYPE.CODE.2	0	0.00

```

# write.csv(collisions_clean, "C:/Users/knigh/Documents/UHD MSDA/CS 5310 Data Mining/Project/collision_clean.csv", row.names = FALSE)
# This is part of our process.

```

Locations Data Cleaning and Geomapping Process Overview

For the analysis of motor vehicle collisions in NYC, only the BOROUGH and ZIP.CODE columns were retained to represent location information. LATITUDE, LONGITUDE, LOCATION, and street-related columns (ON.STREET.NAME, CROSS.STREET.NAME, and OFF.STREET.NAME) were excluded due to their high number of unique values, which would hinder effective modeling. The primary focus is on identifying broader patterns across boroughs rather than hyper-localized street-level analysis.

Handling Missing Location Data

A large number of missing values were present in the BOROUGH and ZIP.CODE columns. To address this, reverse geocoding was performed using ArcGIS to extract addresses, ZIP codes and borough names based on LATITUDE and LONGITUDE. This geocoding process is time-consuming (over 2 hours). To streamline further analysis and report generation, the corresponding code has been commented out, and the most recent output file is used instead.

After geocoding, some rows still lacked ZIP code or BOROUGH information. These cases were manually reviewed and updated using online sources based on available address data. The final version of collisions_borough_zipcode_4.9.25.xlsx includes these manual corrections and is used to supplement missing location values in the main dataset by joining on COLLISION_ID.

Mapping and Cleaning Steps

1. Mapping Missing BOROUGH and ZIP.CODE:

- collisions_clean_4.9.25.xlsx was merged with collisions_borough_zipcode_4.9.25.xlsx using a left join on COLLISION_ID.
- Missing BOROUGH and ZIP.CODE values were filled using the new dataset.
- The updated dataset was saved as collisions_clean_4.9.25_2.xlsx.

2. Final Cleaning:

- Removed unnecessary location columns: LATITUDE, LONGITUDE, LOCATION, ON.STREET.NAME, CROSS.STREET.NAME, and OFF.STREET.NAME.
- Removed rows with:
 - Missing BOROUGH or ZIP.CODE values, or
 - BOROUGH not among the five NYC boroughs: BRONX, MANHATTAN, QUEENS, BROOKLYN, STATEN ISLAND.

3. Final cleaned dataset saved as collisions_clean_4.9.25_3.xlsx.

Install packages

```
# install.packages("readxl")
# install.packages("dplyr")
# install.packages("openxlsx")
# install.packages("writexl")
# install.packages("tidygeocoder")
# install.packages("remotes")
#remotes::install_github("jessecambon/tidygeocoder")
```


Load Necessary Libraries

Part 1

Load Datasets for Extracting Address, BOROUGH, and ZIP.CODE Based on Latitude and Longitude for Missing BOROUGH and ZIP.CODE Values

```
## Load coordinates data
#coordinates <- read_excel("Motor_Vehicle_Collisions-500K.xlsx")

## Reverse Geocode using ArcGIS and extract address components, including ZIP code
#coordinates <- coordinates %>%
# reverse_geocode(lat = LATITUDE, long = LONGITUDE, method = 'arcgis') %>%
# mutate(
#   ZIP.CODE = address_components$zip # Extract ZIP code from address components
# )

## Map County to NYC Borough
#coordinates <- coordinates %>%
# mutate(
#   Borough = case_when(
#     grepl("New York County", address, ignore.case = TRUE) ~ "MANHATTAN",
#     grepl("Kings County", address, ignore.case = TRUE) ~ "BROOKLYN",
#     grepl("Queens County", address, ignore.case = TRUE) ~ "QUEENS",
#     grepl("Bronx County", address, ignore.case = TRUE) ~ "BRONX",
#     grepl("Richmond County", address, ignore.case = TRUE) ~ "STATEN ISLAND",
#     TRUE ~ "Other"
#   )
# )

## Export dataframe with extracted address, boroughs, and ZIP code
#write_xlsx(coordinates, "collisions_borough_zipcode_4.9.25.xlsx")
```

The updated code uses reverse geocoding with ArcGIS to convert latitude and longitude into addresses and extract the ZIP code from the address components. It then maps the county name found in the address to the corresponding NYC borough. The ZIP code is extracted from the `address_components$zip` field, and both the borough and ZIP code are added to the dataset. Finally, the updated dataset, which now includes the address, borough, and ZIP code, is saved as “`collisions_borough_zipcode_4.9.25.xlsx`”.

Part 2

Load Datasets for Mapping Completed BOROUGH and ZIP.CODE

```

## Load datasets
#collisions_clean_4.9.25 <- read_excel("collisions_clean_4.9.25.xlsx")
## This dataset contains cleaned data after removing NAs from NUMBER.OF.PERSONS,
## CONTRIBUTING.FACTOR.VEHICLE, VEHICLE.TYPE.CODE.1, and VEHICLE.TYPE.CODE.2.

#collisions_borough_zipcode_4.9.25 <- read_excel("collisions_borough_zipcode_4.9.25.xlsx")
# This dataset contains completed BOROUGH and ZIP.CODE values for mapping.

# Join datasets based on COLLISION_ID
#merged_collisions <- left_join(collisions_clean_4.9.25,
#                               collisions_borough_zipcode_4.9.25,
#                               by = "COLLISION_ID",
#                               suffix = c("", "_new"))

# Update BOROUGH and ZIP.CODE only where missing
#merged_collisions <- merged_collisions %>%
#  mutate(
#    BOROUGH = ifelse(is.na(BOROUGH), BOROUGH_new, BOROUGH),
#    ZIP.CODE = ifelse(is.na(ZIP.CODE), ZIP.CODE_new, ZIP.CODE)
#  ) %>%
#  select(-BOROUGH_new, -ZIP.CODE_new) # Drop extra columns

# Reorder columns to match original dataset
#merged_collisions <- merged_collisions %>% select(names(collisions_clean_4.9.25))

# Save the final merged dataset
#collisions_clean_4.9.25_2 <- write_xlsx(merged_collisions, "collisions_clean_4.9.25_2.xlsx")

```

Missing BOROUGH and ZIP.CODE values in collision_clean_4.9.25 were filled by mapping information from collisions_borough_zipcode_4.9.25 based on matching COLLISION_ID. The final clean dataset was exported for future use.

Part 3

Remove Unnecessary Columns and Rows from *collisions_clean_4.9.25_2*

Remove columns: LATITUDE, LONGITUDE, LOCATION, ON.STREET.NAME, CROSS.STREET.NAME, OFF.STREET.NAME

Rows Removed: Any rows with missing BOROUGH or ZIP.CODE, OR BOROUGH not in: BRONX, MANHATTAN, QUEENS, BROOKLYN, STATEN ISLAND

```
## Read in cleaned dataset
#collisions_clean_4.9.25_2 <- read_excel("collisions_clean_4.9.25_2.xlsx")

## Step 1: Delete unnecessary columns
#collisions_clean_4.9.25_2 <- collisions_clean_4.9.25_2 %>%
# select(-LATITUDE, -LONGITUDE, -LOCATION,
#        -ON.STREET.NAME, -CROSS.STREET.NAME, -OFF.STREET.NAME)

## Step 2: Filter out rows where:
## - BOROUGH or ZIP.CODE is missing (NA or empty string)
## - BOROUGH is not one of the 5 boroughs

#valid_boroughs <- c("BRONX", "MANHATTAN", "QUEENS", "BROOKLYN", "STATEN ISLAND")

#collisions_clean_4.9.25_2 <- collisions_clean_4.9.25_2 %>%
# filter(!is.na(BOROUGH) & BOROUGH != "",
#        !is.na(ZIP.CODE) & ZIP.CODE != "",
#        BOROUGH %in% valid_boroughs)

# Save the final cleaned dataset
#write_xlsx(collisions_clean_4.9.25_2, "collisions_clean_4.9.25_3.xlsx")
```

Check NA

```
## Read in cleaned dataset
#collisions_clean_4.9.25_3 <- read_excel("collisions_clean_4.9.25_3.xlsx")

## Check the number of NA values in each column of the dataset
#na_count <- colSums(is.na(collisions_clean_4.9.25_3))

## Print the NA counts for each column
#print(na_count)
```

```
collisions_clean = read_xlsx("collisions_clean_4.9.25_3.xlsx")
```

This step concludes the Missing Values. Rebuilding the table from earlier to confirm no NAs remain.

```

num_of_records = nrow(collisions_clean)
num_of_features = ncol(collisions_clean)
data_summary = as.data.frame(matrix(nrow=num_of_features,ncol = 3))

colnames(data_summary) = c("Features", "Missing Values", "% of Missing Values")

data_summary[,1] = colnames(collisions_clean)

for(i in 1:num_of_features){
  data_summary[i,2] = sum(is.na(collisions_clean[,i])) #counting the number of missing values in each cust_info attribute
  data_summary[i,3] = round(100*(sum(is.na(collisions_clean[,i])/num_of_records),2) #turning the count into a percentage
}

print(data_summary)

```

```

##              Features Missing Values % of Missing Values
## 1          CRASH.DATE              0              0
## 2          CRASH.TIME              0              0
## 3          BOROUGH              0              0
## 4          ZIP.CODE              0              0
## 5  NUMBER.OF.PERSONS.INJURED              0              0
## 6  NUMBER.OF.PERSONS.KILLED              0              0
## 7  CONTRIBUTING.FACTOR.VEHICLE.1              0              0
## 8  CONTRIBUTING.FACTOR.VEHICLE.2              0              0
## 9          COLLISION_ID              0              0
## 10         VEHICLE.TYPE.CODE.1              0              0
## 11         VEHICLE.TYPE.CODE.2              0              0

```

Data Inconsistencies

Contributing Factors

This step was completed manually in excel. As it was shown in the Exploratory Analysis above Factor 1 only had 55 unique values. When the geomapping was complete we worked completed these steps on the excel file that was produced. We began extracting just factor 1 and factor 2 into its own spreadsheet and putting the columns ontop of eachother in one column, leaving an empty cell between them to distinguish between the two. Then using conditional formatting we highlighted all duplicate items. There it showed that all values in factor 2 were shared with factor 1. From there we looked through the 55 values and determined they could be bined into 3 groups, unsafe driving, driving conditions, and unspecified. We only kept unspecified because there were to many records with this value to remove. We went through the list and labeled each factor as one of the 3 groups. Then in the full dataset excel file we filtered by the original factors and dragged the new label down through the column. We saved this to the excel sheet below.

```
collisions_clean = read_xlsx("collisions_clean_4.9.25_4.xlsx")
```

Looking at the classes within the FACTOR columns.

The class “none” created by the Handling NA section for Factor 2 was kept, as this means the crashes only had one Factor that caused the accident instead of two.

```
table(collisions_clean$CONTRIBUTING.FACTOR.VEHICLE.1)
```

```
##
## Driving Conditions      Unsafe Driving      Unspecified
##           28340           294452           109928
```

```
table(collisions_clean$CONTRIBUTING.FACTOR.VEHICLE.2)
```

```
##
## Driving Conditions      None      Unsafe Driving      Unspecified
##           6918           94693           43541           287568
```

Looking at the count of ‘unspecified’ in CONTRIBUTING.FACTOR.VEHICLE.2, it accounts for 60% of the column. Because over half of the column does not provide significant information, we will go ahead and remove it.

```
#removing CONTRIBUTING.FACTOR.VEHICLE.2
collisions_clean <- collisions_clean[ , -8]
head
```

```
## function (x, ...)
## UseMethod("head")
## <bytecode: 0x000002183aeb5050>
## <environment: namespace:utils>
```

Vehicle Type

In our exploratory analysis we saw that the most frequency vehicle captured was a sedan. We wanted to take a closer look at any vehicles with sedan listed in the name.

```
sum(grepl("sedan", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 204288
```

```
sum(grepl("sedan", collisions_clean$VEHICLE.TYPE.CODE.2, ignore.case = TRUE))
```

```
## [1] 121967
```

There are 204288 sedans captured in VEHICLE.TYPE.CODE.1 and 121967 in VEHICLE.TYPE.CODE.2. This covers almost 50% of our data.

When viewing the dataset we also saw a lot of Sport Utility Vehicles (SUV) listed.

```
sum(grepl("Sport Utility Vehicle", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 154288
```

```
sum(grepl("Sport Utility Vehicle", collisions_clean$VEHICLE.TYPE.CODE.2, ignore.case = TRUE))
```

```
## [1] 91710
```

There are 154288 SUVs captured in VEHICLE.TYPE.CODE.1 and 91710 in VEHICLE.TYPE.CODE.2. This covers over 35% of our data.

Together sedan and SUVs make up over 80% of our data.

We looked at other vehicles like trucks and motorbikes, but they did not represented a very large portion of the data.

```
#finding trucks
```

```
sum(grepl("truck", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 21685
```

```
sum(grepl("flat bed", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 733
```

```
#Motorcycle and Bike
```

```
sum(grepl("motorcycle", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 3602
```

```
sum(grepl("bike", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 10094
```

```
sum(grepl("moped", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 1773
```

```
sum(grepl("scoot", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 2597
```

```
#Vans
```

```
sum(grepl("van", collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE))
```

```
## [1] 2830
```

Because these other vehicles do not stand out on their own very strong, we decided to put them in another group called others.

So there will be three groups used for modeling - sedans, SUVs, and other.

Converting the classes of VEHICLE.TYPE.CODE.1

```
#VEHICLE.TYPE.CODE.1
#SUV
collisions_clean$VEHICLE.TYPE.CODE.1 <- ifelse(grepl("Sport Utility Vehicle",
  collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE),
  "SUV", collisions_clean$VEHICLE.TYPE.CODE.1)

#sedan
collisions_clean$VEHICLE.TYPE.CODE.1 <- ifelse(grepl("sedan",
  collisions_clean$VEHICLE.TYPE.CODE.1, ignore.case = TRUE),
  "Sedan", collisions_clean$VEHICLE.TYPE.CODE.1)

#Other
collisions_clean$VEHICLE.TYPE.CODE.1 <- ifelse(
  !(collisions_clean$VEHICLE.TYPE.CODE.1 %in% c("SUV", "Sedan")),
  "Other",
  collisions_clean$VEHICLE.TYPE.CODE.1
)
```

Looking at counts and percentages for VEHICLE.TYPE.CODE.1

```
table(collisions_clean$VEHICLE.TYPE.CODE.1)
```

```
##
## Other Sedan SUV
## 74138 204288 154294
```

```
round(prop.table(table(collisions_clean$VEHICLE.TYPE.CODE.1))*100, digits = 1)
```

```
##
## Other Sedan SUV
## 17.1 47.2 35.7
```

Converting the classes of VEHICLE.TYPE.CODE.2

```
#VEHICLE.TYPE.CODE.2
#SUV
collisions_clean$VEHICLE.TYPE.CODE.2 <- ifelse(grepl("Sport Utility Vehicle",
  collisions_clean$VEHICLE.TYPE.CODE.2, ignore.case = TRUE),
  "SUV", collisions_clean$VEHICLE.TYPE.CODE.2)

#sedan
collisions_clean$VEHICLE.TYPE.CODE.2 <- ifelse(grepl("sedan",
  collisions_clean$VEHICLE.TYPE.CODE.2, ignore.case = TRUE),
  "Sedan", collisions_clean$VEHICLE.TYPE.CODE.2)

#Other
collisions_clean$VEHICLE.TYPE.CODE.2 <- ifelse(
  !(collisions_clean$VEHICLE.TYPE.CODE.2 %in% c("SUV", "Sedan", "none")),
  "Other",
  collisions_clean$VEHICLE.TYPE.CODE.2
)
```

Looking at counts and percentages for VEHICLE.TYPE.CODE.2

```
table(collisions_clean$VEHICLE.TYPE.CODE.2)
```

```
##
##   none  Other  Sedan   SUV
## 139312  79727 121967  91714
```

```
round(prop.table(table(collisions_clean$VEHICLE.TYPE.CODE.2))*100, digits = 1)
```

```
##
##   none Other  Sedan   SUV
##   32.2  18.4  28.2  21.2
```

Note that the “none” class created in the NA section is kept here. It means that only 1 vehicle was involved in the crash.

Injured/Killed

For these values we just want to make sure no negatives or decimal numbers.

```
#Looking at the range of the columns
range(collisions_clean$NUMBER.OF.PERSONS.INJURED)
```

```
## [1]  0 34
```

```
range(collisions_clean$NUMBER.OF.PERSONS.KILLED)
```

```
## [1] 0 4
```


No negatives present.

```
any(collisions_clean$NUMBER.OF.PERSONS.INJURED %% 1 != 0)
```

```
## [1] FALSE
```

```
any(collisions_clean$NUMBER.OF.PERSONS.KILLED %% 1 != 0)
```

```
## [1] FALSE
```

No decimals present. So we will move forward with these columns as is.

Zipcode

When researching zipcodes we found that the range of zip codes in New York City primarily covers 10001-104xx and 11004-114xx, and 116xx series throughout the 5 boroughs.

```
range(collisions_clean$ZIP.CODE)
```

```
## [1] 10000 11697
```

We did find 1 Zipcode outside this range, 10000. We confirmed this was not a valid zipcode in the US. For ease of analysis we filtered our excel spreadsheet by this zipcode and found they were all tied to Manhattan. This makes sense as Manhattan starts at 10001. So we will be changing 10000 to 10001.

```
collisions_clean$ZIP.CODE[collisions_clean$ZIP.CODE == 10000] <- 10001
range(collisions_clean$ZIP.CODE)
```

```
## [1] 10001 11697
```

Now the zipcodes appear in the proper range.

Borough

We should have 5 boroughs - Manhattan, Brooklyn, Queens, The Bronx, and Staten Island

```
table(collisions_clean$BOROUGH)
```

```
##
##      BRONX      BROOKLYN      MANHATTAN      QUEENS STATEN ISLAND
##      75581      145694      72015      119125      20305
```

```
round(prop.table(table(collisions_clean$BOROUGH))*100, digits = 1)
```

```
##
##      BRONX      BROOKLYN      MANHATTAN      QUEENS STATEN ISLAND
##      17.5      33.7      16.6      27.5      4.7
```

We can see only the 5 boroughs present. We will move forward as is.

We can also see from the table above that most of these collisions are occurring in Brooklyn, while Staten island has the least amount of crashes.

Date/Time

The data type for our Date and Time is POSIX. We want to keep this data type to further analyze these variables. However POSIX does not allow a time only variable. This is why we are seeing the prefix of 1899-12-31 in front of the times when we import the data. To fix this we found that we need to merge the two columns. To do this we need to get rid of the 1899-12-31, turning the columns into characters eliminates 1899-12-31.

```
head(collisions_clean[ , c("CRASH.DATE","CRASH.TIME")])
```

```
## # A tibble: 6 × 2
##   CRASH.DATE      CRASH.TIME
##   <dtm>          <dtm>
## 1 2024-12-30 00:00:00 1899-12-31 12:30:00
## 2 2024-12-29 00:00:00 1899-12-31 06:02:00
## 3 2024-12-29 00:00:00 1899-12-31 20:00:00
## 4 2024-12-29 00:00:00 1899-12-31 22:10:00
## 5 2024-12-29 00:00:00 1899-12-31 03:04:00
## 6 2024-12-29 00:00:00 1899-12-31 17:28:00
```

```
collisions_clean$CRASH.TIME <- format(collisions_clean$CRASH.TIME, "%H:%M:%S")
collisions_clean$CRASH.DATE <- format(collisions_clean$CRASH.DATE, "%Y-%m-%d")
head(collisions_clean[ , c("CRASH.DATE","CRASH.TIME")])
```

```
## # A tibble: 6 × 2
##   CRASH.DATE CRASH.TIME
##   <chr>      <chr>
## 1 2024-12-30 12:30:00
## 2 2024-12-29 06:02:00
## 3 2024-12-29 20:00:00
## 4 2024-12-29 22:10:00
## 5 2024-12-29 03:04:00
## 6 2024-12-29 17:28:00
```

We can see the 1899-12-31 has now been remove. Now to merge and change back to POSIX

```
collisions_clean$CRASH.DATETIME <- as.POSIXct(
  paste(collisions_clean$CRASH.DATE, collisions_clean$CRASH.TIME),
  format = "%Y-%m-%d %H:%M:%S",
  tz = "UTC"
)
head(collisions_clean[ , c("CRASH.DATE","CRASH.TIME","CRASH.DATETIME")])
```

```
## # A tibble: 6 × 3
##   CRASH.DATE CRASH.TIME CRASH.DATETIME
##   <chr>      <chr>      <dtm>
## 1 2024-12-30 12:30:00 2024-12-30 12:30:00
## 2 2024-12-29 06:02:00 2024-12-29 06:02:00
## 3 2024-12-29 20:00:00 2024-12-29 20:00:00
## 4 2024-12-29 22:10:00 2024-12-29 22:10:00
## 5 2024-12-29 03:04:00 2024-12-29 03:04:00
## 6 2024-12-29 17:28:00 2024-12-29 17:28:00
```

We can see that the date and times are now merged and match the initial columns, so now we will remove the separate date and time columns.

```
collisions_clean <- collisions_clean[, -(1:2)]
```

This concludes the data inconsistency section. All variables have been checked. We have binned columns with large amounts of unique variables. We have merged columns that to maintain proper data types. As well as checked numeric values to make sure no negatives or decimals present.

Feature Engineering

In our analysis we would like to predict crash severity. To do this we need to create labels that specify how severe a crash is. Currently we just have numbers of how many people were effected.

To create our labels, we want to make 3 classes of severity - Low, Moderate, and High.

Low will consist of no injuries and no fatalities.

Moderate will consist of 1 to 3 injuries and no fatalities.

High will consist of more than 3 injuries and at least one fatality.

Creating the code below as an IF statement, showing that if there are no injuries or fatalities to fall in the Low bin, if 1-3 injuries but no fatalities fall in the Moderate bin, and if there are more than 3 injuries and/or fatalities greater than 0, fall in the High bin.

```
collisions_clean$SEVERITY.CLASS <- ifelse(
  collisions_clean$NUMBER.OF.PERSONS.INJURED == 0 & collisions_clean$NUMBER.OF.PERSONS.KILLED ==
0, "Low",
  ifelse(
    collisions_clean$NUMBER.OF.PERSONS.INJURED >= 1 & collisions_clean$NUMBER.OF.PERSONS.INJURED
<= 3 & collisions_clean$NUMBER.OF.PERSONS.KILLED == 0, "Moderate",
    "High"
  )
)
```

Checking the class balance of our labels. As we saw in the exploratory analysis, we have a high number of crashes where no one was effected, and the injury levels are generally on the lower side. Because of this we do expect some class imbalance.

```
table(collisions_clean$SEVERITY.CLASS)
```

```
##
##      High      Low Moderate
##      5131    277573    150016
```

```
round(prop.table(table(collisions_clean$SEVERITY.CLASS))*100, digits = 1)
```

```
##
##      High      Low Moderate
##      1.2    64.1    34.7
```

This is about what we expected. About 64% of our crashes are low risk, 35% is moderate risk, and only 1% is high risk.

We will be considering these class imbalances in our next steps to create training and testing sets for our models. We will be utilizing strategic sampling methods to best level out all classes.

Now we have clean data and ready for next steps in our analysis.

Further Exploration

Now that we have clean data, we want to analyze some of our features further. In the next steps we will tackle imbalanced data by stratified sampling long with under sampling of the larger classes of Low and Moderate to match the amount of High records. Since this will cause further reduction in records, we want to look at additional information before doing so.

Time analysis

Breaking down the time variable to further analyze.

```
## package 'lubridate' successfully unpacked and MD5 sums checked
```

```
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
# Extract parts of the date
collisions_Time <- collisions_clean
collisions_Time$year <- year(collisions_clean$CRASH.DATETIME)
collisions_Time$month <- month(collisions_clean$CRASH.DATETIME, label = TRUE)
collisions_Time$weekday <- wday(collisions_clean$CRASH.DATETIME, label = TRUE)
collisions_Time$hour <- hour(collisions_clean$CRASH.DATETIME)
```

Counting the amount of collisions per time frame

```
# Collisions per year
yearly_counts <- count(collisions_Time, year) #uses dplyr and creates a separate df with two columns

# Collisions per weekday
weekday_counts <- count(collisions_Time, weekday)

# Collisions per month
monthly_counts <- count(collisions_Time, month)

#collisions per hour
hourly_counts <- count(collisions_Time, hour) #between 4pm and 5pm is when most crashes occur

print(yearly_counts)
```

```
## # A tibble: 5 × 2
##   year      n
##   <dbl> <int>
## 1  2020 102017
## 2  2021 102200
## 3  2022  95102
## 4  2023  88754
## 5  2024  44647
```

```
print(weekday_counts)
```

```
## # A tibble: 7 × 2
##   weekday      n
##   <ord>   <int>
## 1 Sun     56276
## 2 Mon     60736
## 3 Tue     61350
## 4 Wed     61603
## 5 Thu     63133
## 6 Fri     68072
## 7 Sat     61550
```

```
print(monthly_counts)
```

```
## # A tibble: 12 × 2
##   month      n
##   <ord> <int>
## 1 Jan    41233
## 2 Feb    38340
## 3 Mar    40544
## 4 Apr    33309
## 5 May    39330
## 6 Jun    40164
## 7 Jul    35816
## 8 Aug    33657
## 9 Sep    33725
## 10 Oct    34474
## 11 Nov    31418
## 12 Dec    30710
```

```
print(hourly_counts)
```

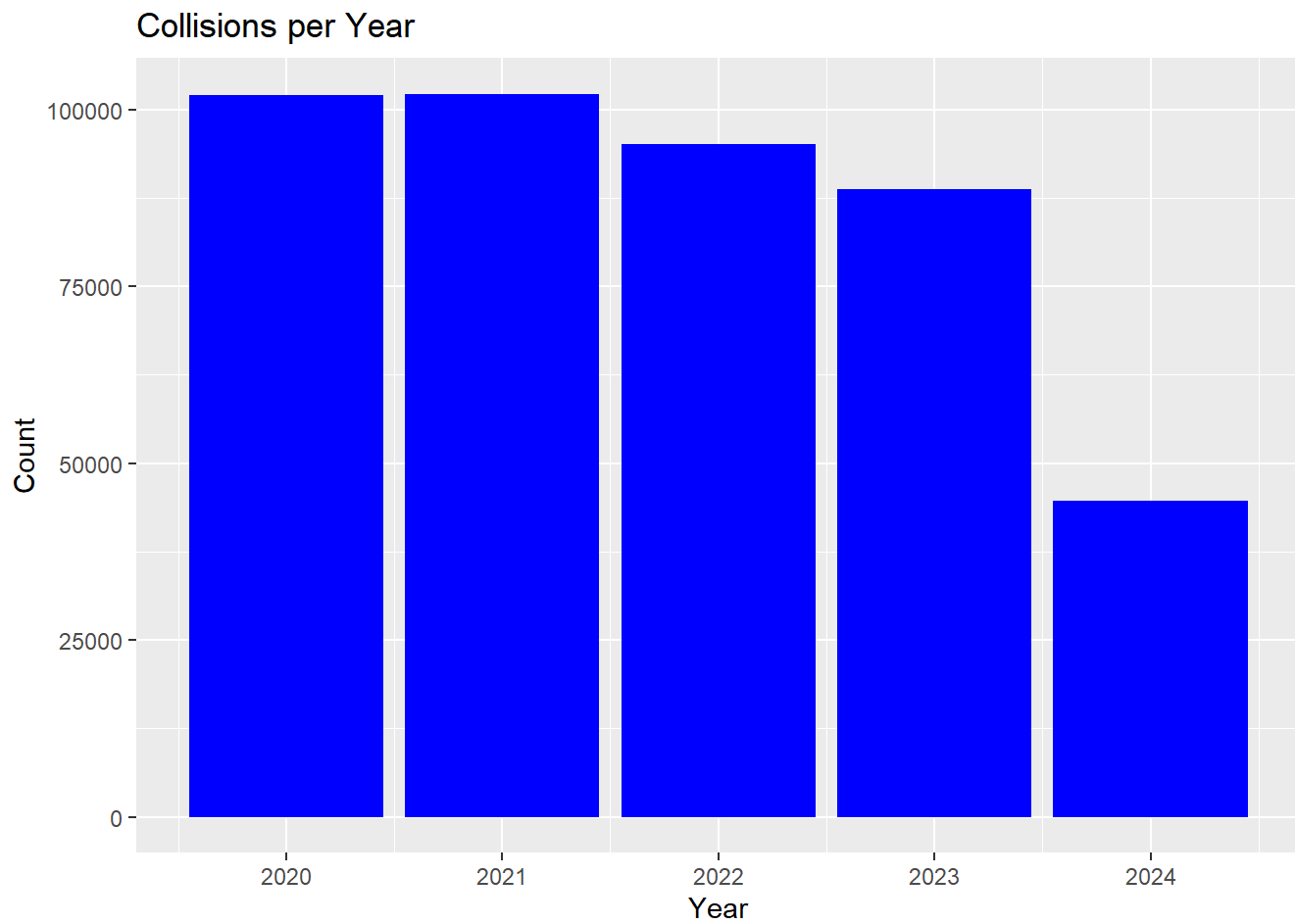
```
## # A tibble: 24 × 2
##   hour      n
##   <int> <int>
## 1     0 18931
## 2     1 10209
## 3     2  8036
## 4     3  7339
## 5     4  7856
## 6     5  8141
## 7     6 11296
## 8     7 14858
## 9     8 21718
## 10    9 19702
## # i 14 more rows
```

Graphing the counts

```
library(ggplot2)
```

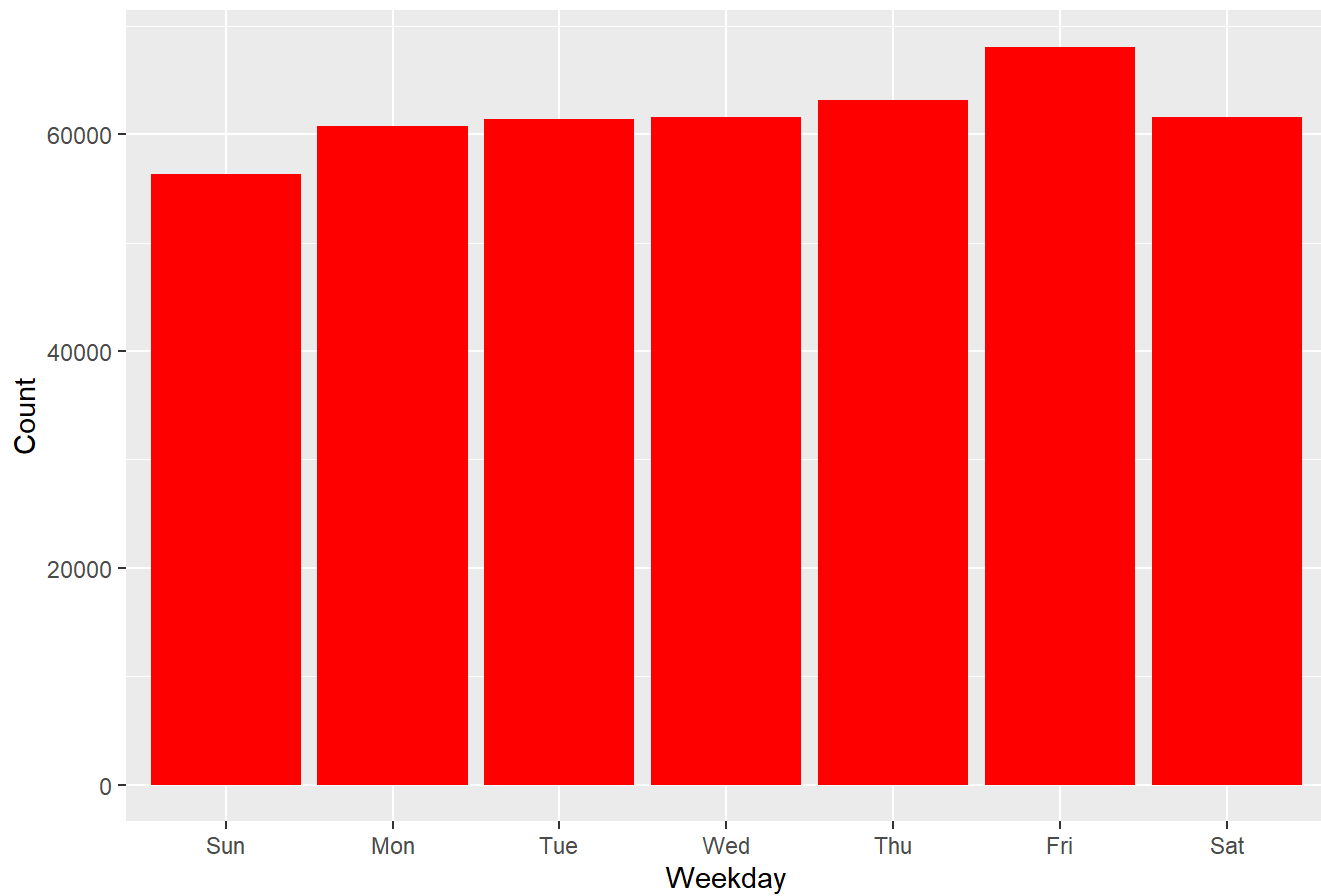
```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
# By year
ggplot(yearly_counts, aes(x = year, y = n)) +
  geom_col(fill = "blue") +
  labs(title = "Collisions per Year", y = "Count", x = "Year")
```

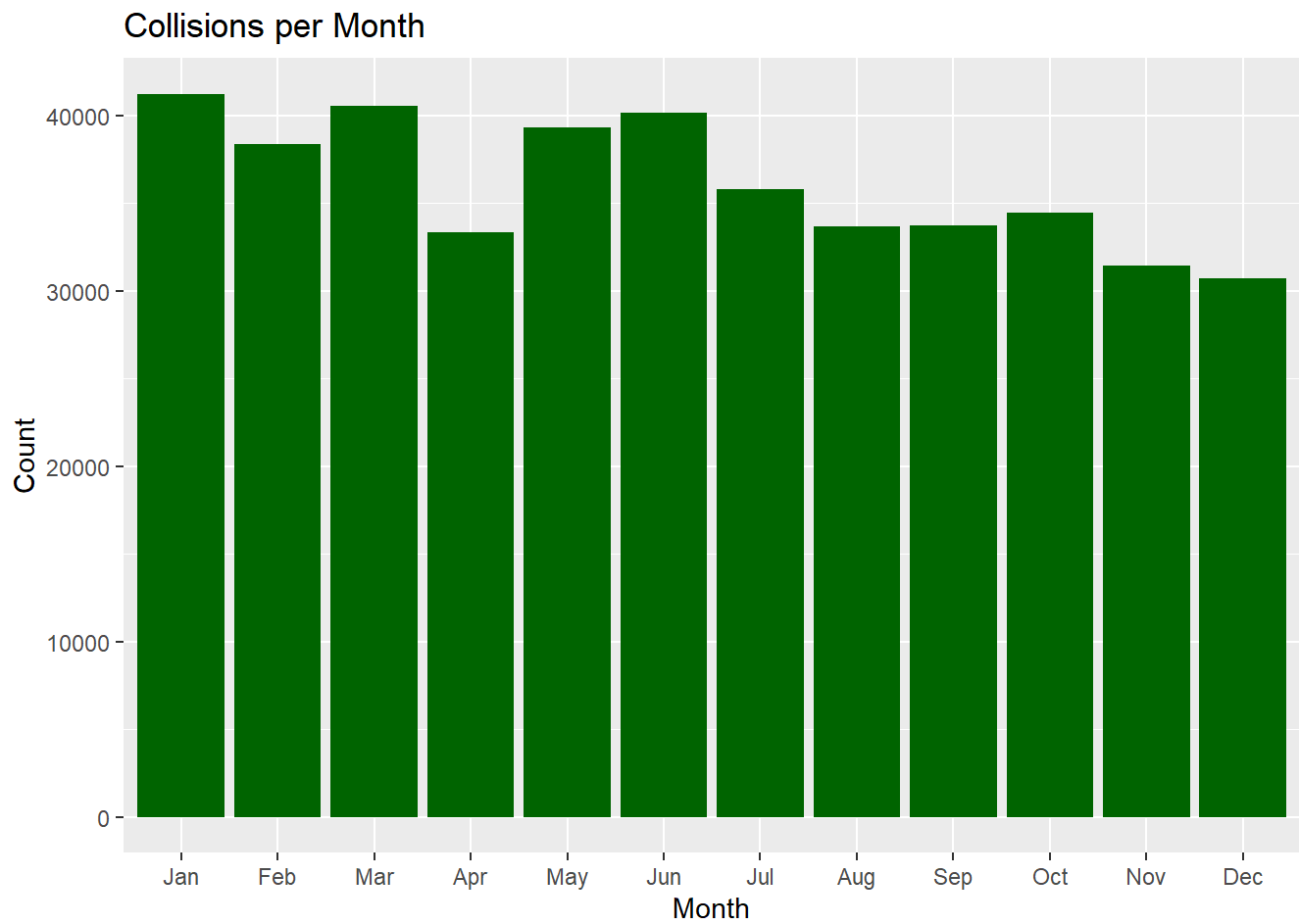


```
# By weekday  
ggplot(weekday_counts, aes(x = weekday, y = n)) +  
  geom_col(fill = "red") +  
  labs(title = "Collisions by Day of the Week", y = "Count", x = "Weekday")
```

Collisions by Day of the Week

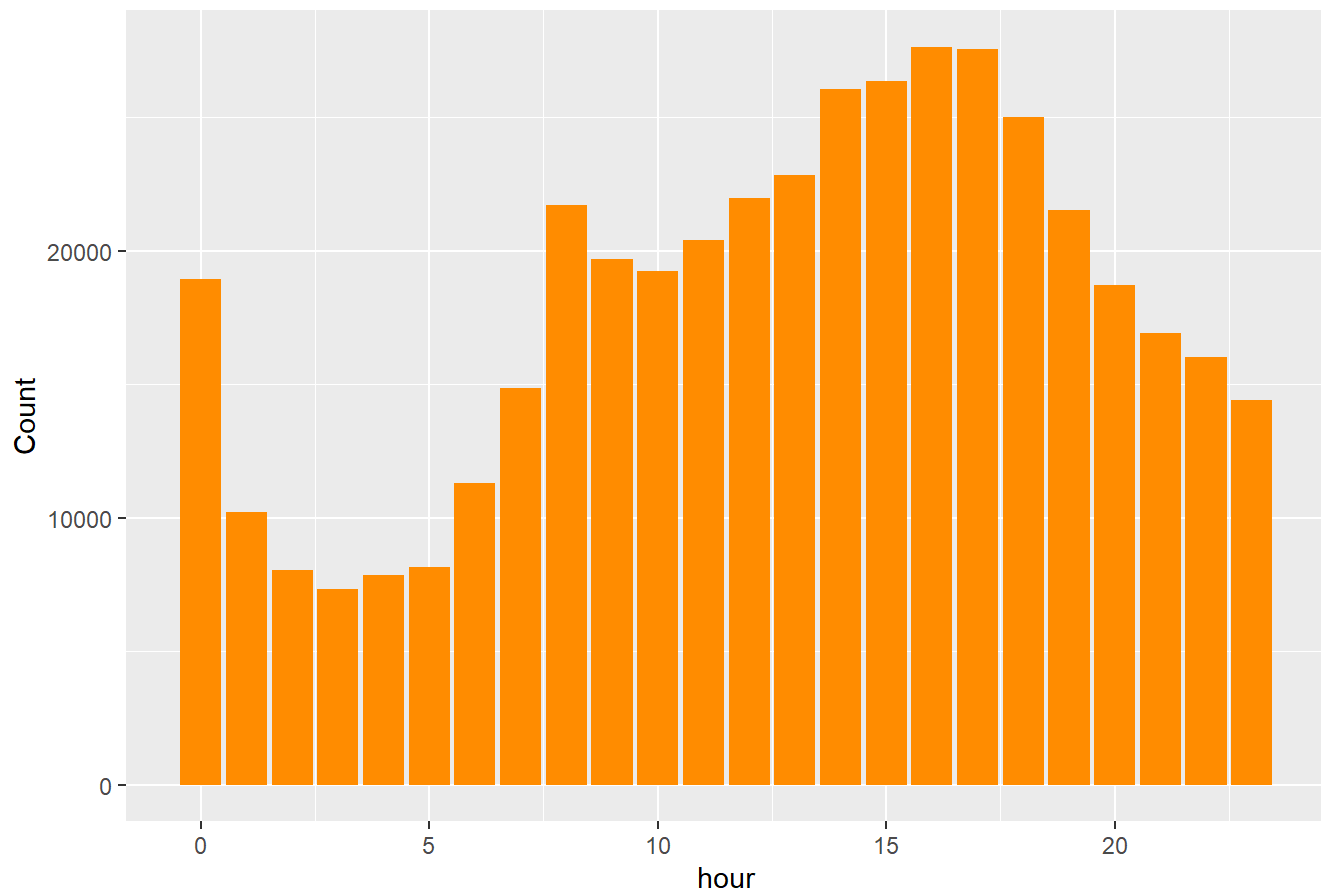


```
# By month
ggplot(monthly_counts, aes(x = month, y = n)) +
  geom_col(fill = "darkgreen") +
  labs(title = "Collisions per Month", y = "Count", x = "Month")
```

```
#by hour  
ggplot(hourly_counts, aes(x = hour, y = n)) +  
  geom_col(fill = "darkorange") +  
  labs(title = "General Collisions per hour", y = "Count", x = "hour")
```

General Collisions per hour

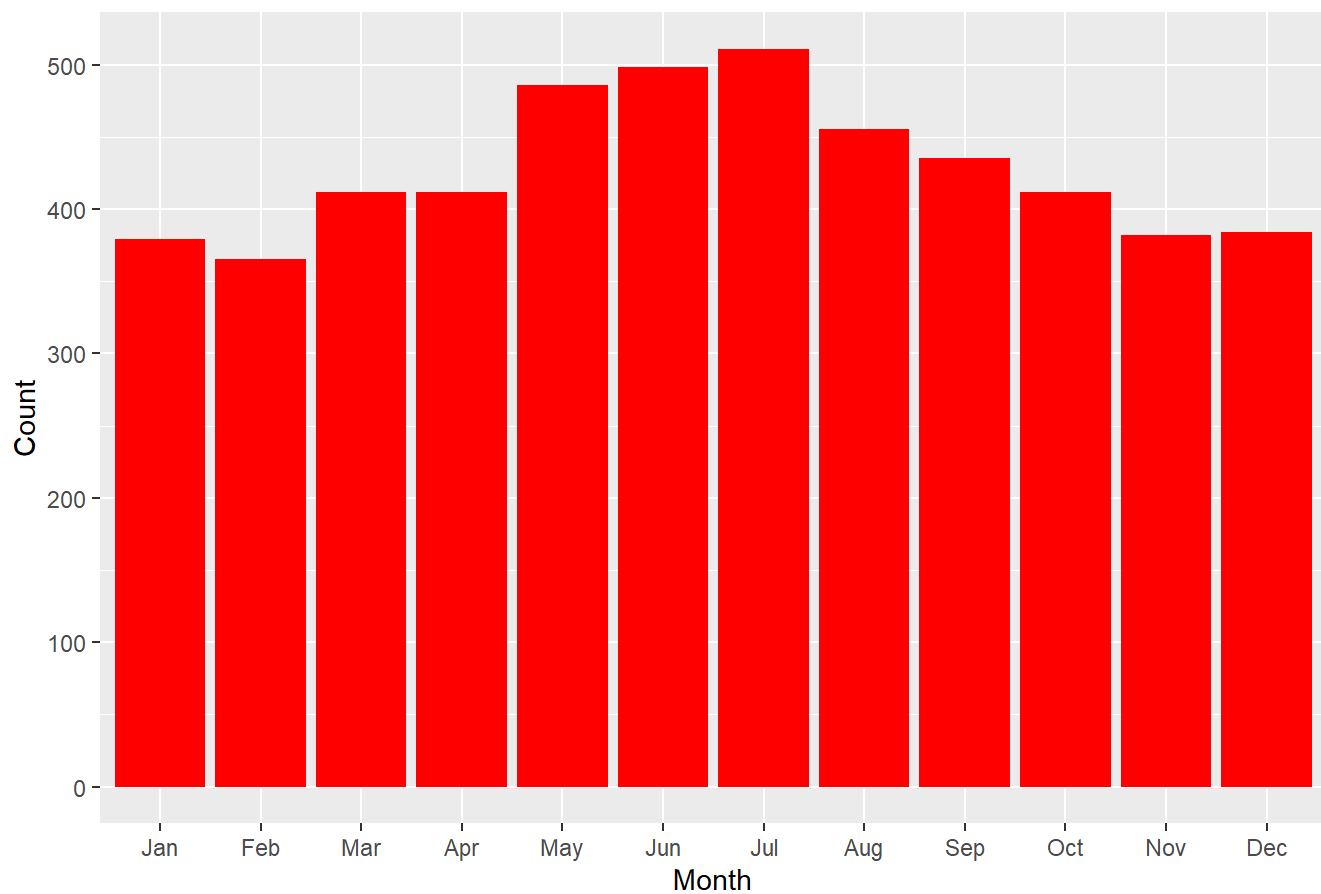


Now to see when specifically High severity collisions occur

```
#filtering only the High classes to analyze
High_Time <- filter(collisions_Time, SEVERITY.CLASS == "High")

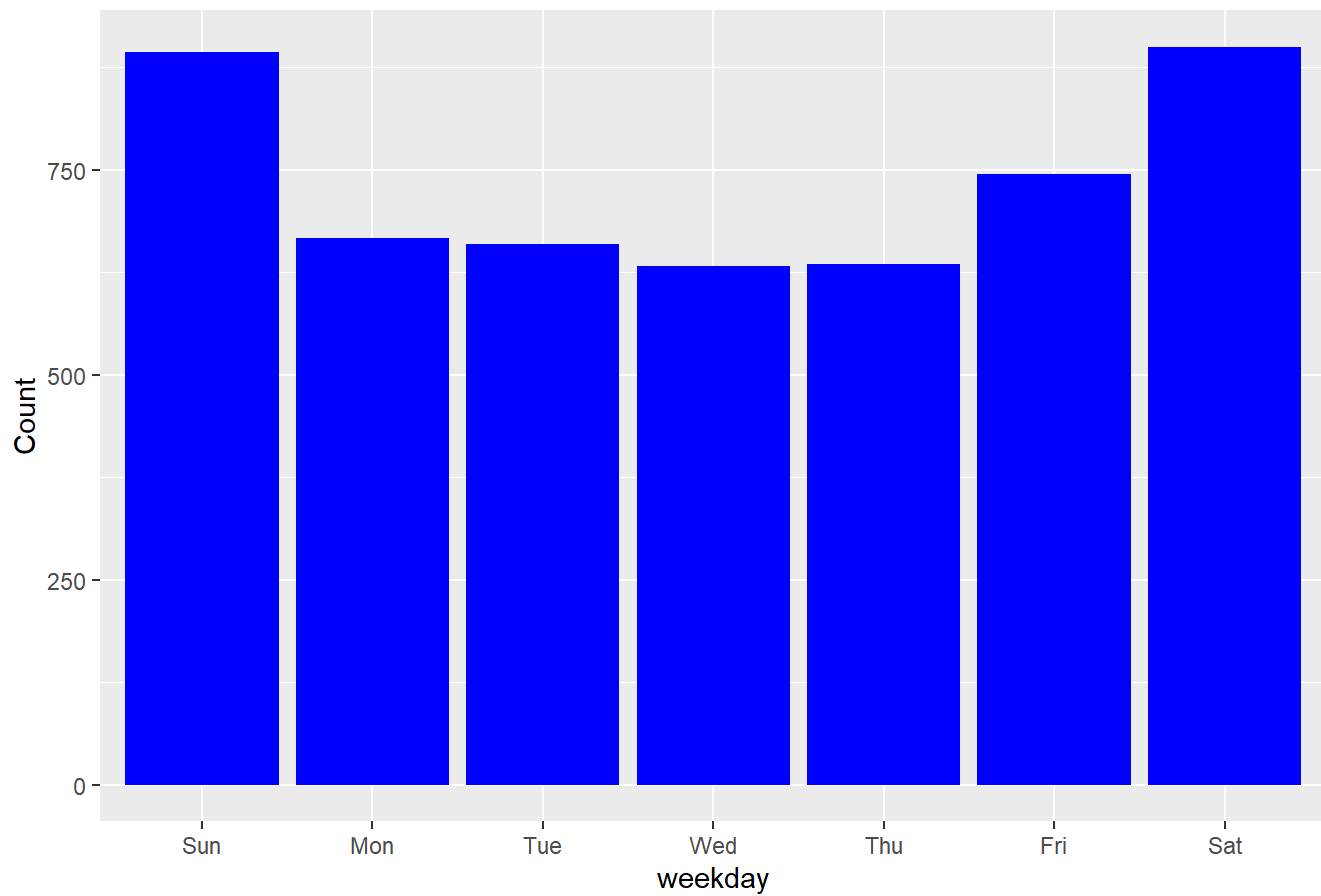
#month
ggplot(High_Time, aes(x = month)) +
  geom_bar(fill = "red") +
  labs(title = "High Severity Collisions by Month", x = "Month", y = "Count")
```

High Severity Collisions by Month



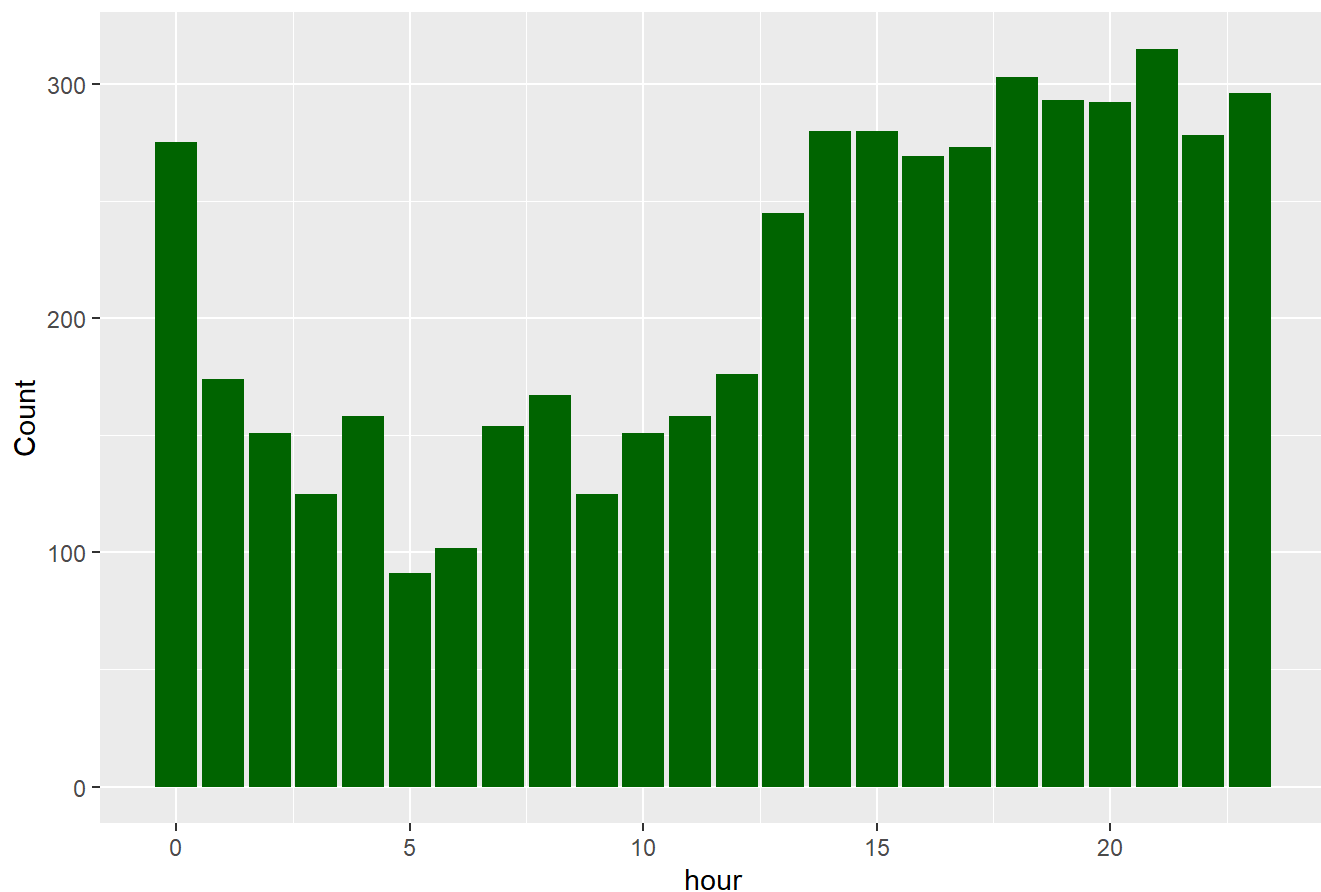
```
#weekday  
ggplot(High_Time, aes(x = weekday)) +  
  geom_bar(fill = "blue") +  
  labs(title = "High Severity Collisions by weekday", x = "weekday", y = "Count")
```

High Severity Collisions by weekday



```
#hour
ggplot(High_Time, aes(x = hour)) +
  geom_bar(fill = "darkgreen") +
  labs(title = "High Severity Collisions by hour", x = "hour", y = "Count")
```

High Severity Collisions by hour



```
#Trend of high severity crashes over time
yearly_High_counts <- count(High_Time, year)
```

```
ggplot(yearly_High_counts, aes(x = year, y = n)) +
  geom_line(color = "red") +
  geom_smooth(color = "blue") +
  labs(title = "High Severity Collisions Over Time",
       x = "Year",
       y = "Count")
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

```
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : span too small. fewer data values than degrees of freedom.
```

```
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : pseudoinverse used at 2020
```

```
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
## : neighborhood radius 2.02
```

```
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,  
## : reciprocal condition number 0
```

```
## Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,  
## : There are other near singularities as well. 4.0804
```

```
## Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x  
## else if (is.data.frame(newdata))  
## as.matrix(model.frame(delete.response(terms(object))), : span too small. fewer  
## data values than degrees of freedom.
```

```
## Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x  
## else if (is.data.frame(newdata))  
## as.matrix(model.frame(delete.response(terms(object))), : pseudoinverse used at  
## 2020
```

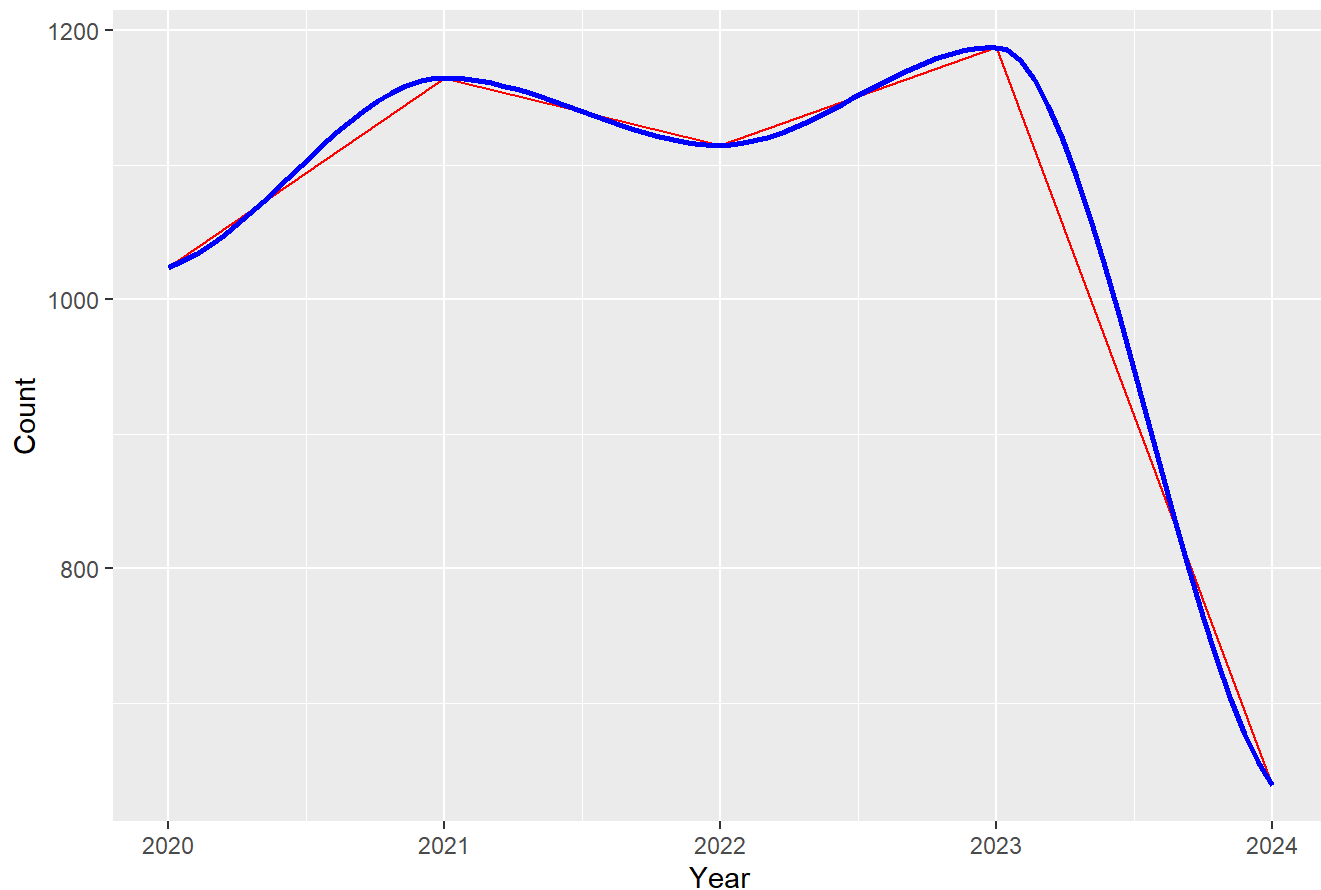
```
## Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x  
## else if (is.data.frame(newdata))  
## as.matrix(model.frame(delete.response(terms(object))), : neighborhood radius  
## 2.02
```

```
## Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x  
## else if (is.data.frame(newdata))  
## as.matrix(model.frame(delete.response(terms(object))), : reciprocal condition  
## number 0
```

```
## Warning in predLoess(object$y, object$x, newx = if (is.null(newdata)) object$x  
## else if (is.data.frame(newdata))  
## as.matrix(model.frame(delete.response(terms(object))), : There are other near  
## singularities as well. 4.0804
```

```
## Warning in max(ids, na.rm = TRUE): no non-missing arguments to max; returning  
## -Inf
```

High Severity Collisions Over Time



Vehicle Type

What type of cars were most common in total collisions

```
round(prop.table(table(collisions_clean$VEHICLE.TYPE.CODE.1))*100, digits = 1)
```

```
##
## Other Sedan   SUV
## 17.1  47.2  35.7
```

```
round(prop.table(table(collisions_clean$VEHICLE.TYPE.CODE.2))*100, digits = 1)
```

```
##
## none Other Sedan   SUV
## 32.2  18.4  28.2  21.2
```

What type of cars were most involved with High severity collisions

```
High_collisions <- filter(collisions_clean, SEVERITY.CLASS == "High")
round(prop.table(table(High_collisions$VEHICLE.TYPE.CODE.1))*100, digits = 1)
```

```
##
## Other Sedan    SUV
## 17.8  46.3  35.9
```

```
round(prop.table(table(High_collisions$VEHICLE.TYPE.CODE.2))*100, digits = 1)
```

```
##
## none Other Sedan    SUV
## 15.3  13.5  39.5  31.7
```

Contributing Factors

Looking at the table of contributing factors for both total collisions and high severity collisions

```
#total collisions
round(prop.table(table(collisions_clean$CONTRIBUTING.FACTOR.VEHICLE.1))*100, digits = 1)
```

```
##
## Driving Conditions      Unsafe Driving      Unspecified
##           6.5           68.0           25.4
```

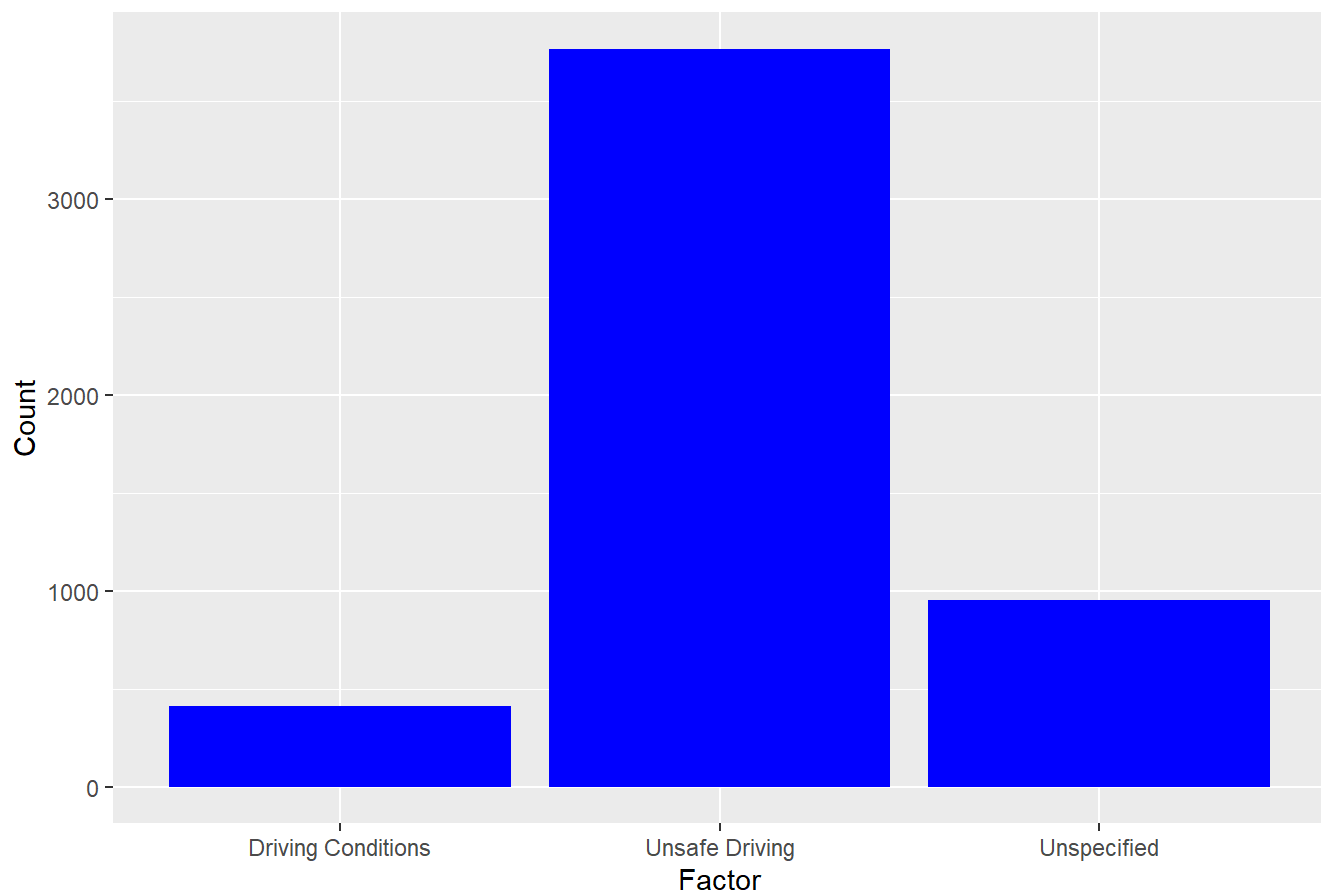
```
#high severity collisions
round(prop.table(table(High_collisions$CONTRIBUTING.FACTOR.VEHICLE.1))*100, digits = 1)
```

```
##
## Driving Conditions      Unsafe Driving      Unspecified
##           8.0           73.4           18.6
```

Visualizing the data

```
ggplot(High_collisions, aes(x = CONTRIBUTING.FACTOR.VEHICLE.1)) +
  geom_bar(fill = "blue") +
  labs(title = "High Severity Collisions by Contributing Factor", x = "Factor", y = "Count")
```


High Severity Collisions by Contributing Factor



Handling the imbalanced data

As seen in the above section as well as our exploratory analysis we saw that our target variable is heavily imbalanced. Keeping this imbalance will likely through off future modeling. So we plan to stratify and under-sample to create our modeling data.

We will keep all High severity crash records (5131), and bring the other classes, Low and Moderate, down to match the amount of records we have for High.

Beginning with pulling the classes apart to stratify them individually creating a more simple and easy to follow processes.

```
#splitting the classes their own dataframe
high_class  <- collisions_clean %>% filter(SEVERITY.CLASS == "High")
moderate_class <- collisions_clean %>% filter(SEVERITY.CLASS == "Moderate")
low_class    <- collisions_clean %>% filter(SEVERITY.CLASS == "Low")
```

Beginning the under-sampling of Moderate and Low classes. The selection of records will be done randomly, 5131 of each will be picked.

```
# Sampling Moderate and Low classes
set.seed(123) # for reproducibility
moderate_sample <- moderate_class %>% sample_n(5131)
low_sample      <- low_class %>% sample_n(5131)
```

Now that the samples have been collected, we will combine them all back together.

```
# Combining classes into a new balanced dataset for modeling
collisions_balanced <- bind_rows(high_class, moderate_sample, low_sample)
```

Checking class balance of our model data.

```
table(collisions_balanced$SEVERITY.CLASS)
```

```
##
##      High      Low Moderate
##      5131      5131      5131
```

We can see all classes now have the same amount of records!

Checking the balanced data's context

We created the new modeling data by undersampling the majority classes randomly. We want to make sure that the new dataset holds about the same information as our original cleaned dataset.

Reviewing the overall summary

```
summary(collisions_balanced)
```

```
##      BOROUGH      ZIP.CODE      NUMBER.OF.PERSONS.INJURED
## Length:15393      Min.       :10001      Min.       : 0.000
## Class :character  1st Qu.:10456      1st Qu.: 0.000
## Mode  :character  Median :11209      Median : 1.000
##                                     Mean  :10902      Mean   : 1.701
##                                     3rd Qu.:11354      3rd Qu.: 4.000
##                                     Max.   :11697      Max.   :34.000
## NUMBER.OF.PERSONS.KILLED CONTRIBUTING.FACTOR.VEHICLE.1 COLLISION_ID
## Min.       :0.00000      Length:15393      Min.       :4267434
## 1st Qu.:0.00000      Class :character  1st Qu.:4399112
## Median :0.00000      Mode  :character  Median :4516952
## Mean      :0.07153                                     Mean   :4513990
## 3rd Qu.:0.00000                                     3rd Qu.:4632735
## Max.       :4.00000                                     Max.   :4782918
## VEHICLE.TYPE.CODE.1 VEHICLE.TYPE.CODE.2 CRASH.DATETIME
## Length:15393      Length:15393      Min.       :2020-01-01 00:37:00.00
## Class :character  Class :character  1st Qu.:2021-03-12 06:20:00.00
## Mode  :character  Mode  :character  Median :2022-04-02 23:50:00.00
##                                     Mean   :2022-04-06 20:15:59.23
##                                     3rd Qu.:2023-05-23 16:29:00.00
##                                     Max.   :2024-12-29 22:12:00.00
## SEVERITY.CLASS
## Length:15393
## Class :character
## Mode  :character
##
##
##
```

Boroughs

Comparing borough class percentages between original cleaned data and balanced data.

```
#original dataset
round(prop.table(table(collisions_clean$BOROUGH))*100, digits = 1)
```

##					
##	BRONX	BROOKLYN	MANHATTAN	QUEENS	STATEN ISLAND
##	17.5	33.7	16.6	27.5	4.7

```
#new dataset
round(prop.table(table(collisions_balanced$BOROUGH))*100, digits = 1)
```

##					
##	BRONX	BROOKLYN	MANHATTAN	QUEENS	STATEN ISLAND
##	18.3	34.1	14.9	27.8	4.8

Both datasets have roughly the same distribution of borough classes.

Zipcode

Looking at the range of both datasets.

```
#original dataset
range(collisions_clean$ZIP.CODE)
```

```
## [1] 10001 11697
```

```
#new dataset
range(collisions_balanced$ZIP.CODE)
```

```
## [1] 10001 11697
```

Same range between both datasets.

Looking at the number of unique values.

```
sum(n_distinct(collisions_balanced$ZIP.CODE))
```

```
## [1] 193
```

After some consideration for our planned models, there might be too many unique variables for kNN, Decision Tree, and Random Forest. We will keep this in mind going forward, possibly remove if needed.

Number of persons injured or killed

Since our target variable is derived from these two columns, we will remove them to avoid redundancy and prevent the model from simply learning their direct relationship, rather than underlying patterns.

```
collisions_balanced <- collisions_balanced[ , -(3:4)]
```

columns NUMBER.OF.PERSONS.INJURED and NUMBER.OF.PERSONS.KILLED have been removed.

Contributing factor

Like the borough column we want to make sure the proportions match between datasets.

```
#original data
round(prop.table(table(collisions_clean$CONTRIBUTING.FACTOR.VEHICLE.1))*100, digits = 1)
```

```
##
## Driving Conditions      Unsafe Driving      Unspecified
##              6.5              68.0              25.4
```

```
#new data
round(prop.table(table(collisions_balanced$CONTRIBUTING.FACTOR.VEHICLE.1))*100, digits = 1)
```

```
##
## Driving Conditions      Unsafe Driving      Unspecified
##              7.0              70.8              22.2
```

The proportions between the datasets are very similar.

Collision ID

This has been kept to help if we needed to connect to any records for any reason, like how we did for the Geodata. However, we do not expect to need it again in the future as we are about to model. IDs do not offer much information in modeling since every value is unique.

```
collisions_balanced <- collisions_balanced[, -4]
```

Collision ID has been removed.

Vehicle types

we want to make sure the proportion of these classes match between datasets as well.

Starting with Vehicle one

```
#original data
round(prop.table(table(collisions_clean$VEHICLE.TYPE.CODE.1))*100, digits = 1)
```

```
##
## Other Sedan   SUV
##  17.1  47.2  35.7
```

```
#new data
round(prop.table(table(collisions_balanced$VEHICLE.TYPE.CODE.1))*100, digits = 1)
```

```
##
## Other Sedan   SUV
##  18.0  46.0  35.9
```

Proportions are about the same between datasets.

DATE/TIME

Looking to see the new dataset covers roughly the same time as the original.

```
#original dataset
summary(collisions_balanced$CRASH.DATETIME)
```

```
##                Min.                1st Qu.
## "2020-01-01 00:37:00.0000" "2021-03-12 06:20:00.0000"
##                Median                Mean
## "2022-04-02 23:50:00.0000" "2022-04-06 20:15:59.2321"
##                3rd Qu.                Max.
## "2023-05-23 16:29:00.0000" "2024-12-29 22:12:00.0000"
```

```
#new dataset
summary(collisions_clean$CRASH.DATETIME)
```

```
##                Min.                1st Qu.
## "2020-01-01 00:00:00.0000" "2021-01-28 18:23:00.0000"
##                Median                Mean
## "2022-02-20 13:31:30.0000" "2022-03-07 02:26:09.8121"
##                3rd Qu.                Max.
## "2023-04-20 22:33:15.0000" "2024-12-30 12:30:00.0000"
```

Both datasets cover the full 4 years. Both Min values are 1/1/202. 1st quarters are both in 2021 only two months difference. Medians are both 2022 close in dates. Means are both in 2022 only 1 month apart. 3rd quarters also both in 2023 and only 1 month apart. Last Max of both are only one day difference.

We believe this new balanced dataset provided similar information as our original cleaned dataset, just on a smaller scale.

Creating training, validation, and testing sets

We want to randomly sample again to create our training, validation, and testing sets. We will divide the data 70% training, 15% validation, and 15% testing.

Taking a similar approach like we did in lab 8 to randomly sample the data.

```
#randomly divide the data into a 70/30 split. The 30 will be spilt more later
num_of_rows <- nrow(collisions_balanced)
collisions.train.num_rows <- num_of_rows*.70
set.seed(123)
collisions.sample <- sample(num_of_rows, collisions.train.num_rows)
```

Defining the Training set

```
# 70% training data
collisions_train <- collisions_balanced[collisions.sample, ]

remaining_TV <- collisions_balanced[-collisions.sample, ]
```

Splitting the remaining data into validation and testing

```
#splitting the remaining 30% further into validation and testing
num_of_rows <- nrow(remaining_TV)
collisions.val.num_rows <- num_of_rows*.50
set.seed(123)
collisions.sample2 <- sample(num_of_rows, collisions.val.num_rows)
```

Defining the Validation and Testing set.

```
# 15% validation data
collisions_validation <- remaining_TV[collisions.sample2, ]

# 15% testing data
collisions_test <- remaining_TV[-collisions.sample2, ]
```

Checking the proportions of the target class in these sets.

```
#training
round(prop.table(table(collisions_train$SEVERITY.CLASS))*100, digits = 1)
```

```
##
##      High      Low Moderate
##      33.5      33.3      33.2
```

```
#validation
round(prop.table(table(collisions_validation$SEVERITY.CLASS))*100, digits = 1)
```

```
##
##      High      Low Moderate
##      32.9      33.5      33.6
```

```
#testing
round(prop.table(table(collisions_test$SEVERITY.CLASS))*100, digits = 1)
```

```
##
##      High      Low Moderate
##      32.8      33.5      33.7
```

The proportions of classes are very well distributed among the sets. We are ready for modeling. :)

Modeling

kNN

To begin kNN we need to normalize the data.

We will be turning the categorical variables into dummy variables, except for Zipcode.

After some research we found that ZIP.CODE will be quite hard for kNN to process since it has 193 unique values. The two main options we saw were dummy variables or frequency of occurrence, other steps would require additional processing and possibly modeling for zipcodes. Since dummy variables would create 193 columns, frequency would assume a continuous/ordinal relationship, and we still have location information from borough, we have decided to drop zipcode for kNN.

```
#dropping ZIP.CODE
collisions_train_kNN <- collisions_train[ , -2]
collisions_validation_kNN <- collisions_validation[ , - 2]
collisions_test_kNN <- collisions_test[ , - 2]
```

Turning other category variables into dummy variables.

```
## package 'fastDummies' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
#Borough
collisions_train_kNN <- dummy_cols(collisions_train_kNN, select_columns = "BOROUGH")
collisions_validation_kNN <- dummy_cols(collisions_validation_kNN, select_columns = "BOROUGH")
collisions_test_kNN <- dummy_cols(collisions_test_kNN, select_columns = "BOROUGH")

#Contributing factor
collisions_train_kNN <- dummy_cols(collisions_train_kNN, select_columns = "CONTRIBUTING.FACTOR.VEHICL
E.1")
collisions_validation_kNN <- dummy_cols(collisions_validation_kNN, select_columns = "CONTRIBUTIN
G.FACTOR.VEHICLE.1")
collisions_test_kNN <- dummy_cols(collisions_test_kNN, select_columns = "CONTRIBUTING.FACTOR.VEH
ICLE.1")

#Vehicle type 1
collisions_train_kNN <- dummy_cols(collisions_train_kNN, select_columns = "VEHICLE.TYPE.CODE.1")
collisions_validation_kNN <- dummy_cols(collisions_validation_kNN, select_columns = "VEHICLE.TYP
E.CODE.1")
collisions_test_kNN <- dummy_cols(collisions_test_kNN, select_columns = "VEHICLE.TYPE.CODE.1")

#vehicle type 2
collisions_train_kNN <- dummy_cols(collisions_train_kNN, select_columns = "VEHICLE.TYPE.CODE.2")
collisions_validation_kNN <- dummy_cols(collisions_validation_kNN, select_columns = "VEHICLE.TYP
E.CODE.2")
collisions_test_kNN <- dummy_cols(collisions_test_kNN, select_columns = "VEHICLE.TYPE.CODE.2")
```

All dummy variables have been created

For date/time data we will be turning this variable into a numeric type. When converting POSIX into numeric, it gives us a big number that represents that specific time-stamp. Specifically, it counts the number of seconds from POSIX's start date of 1/1/1970. By using this method it creates a natural scale.

Since we have 3 sets, training, validation, and testing we need to make sure they are all on the same scale. Thankfully we only have 1 numeric variable to worry about. Since we have dummy variables with values of either 0 or 1, we will use the min/max method to have all columns stay within the range. This ensures that the normalization across all sets is consistent. Using the training set's range also helps the model handle any values in the validation or test sets that fall outside of the training range.

```
#Coverting the date/time to numeric (seconds)
collisions_train_kNN$CRASH.DATETIME <- as.numeric(collisions_train_kNN$CRASH.DATETIME)
collisions_validation_kNN$CRASH.DATETIME <- as.numeric(collisions_validation_kNN$CRASH.DATETIME)
collisions_test_kNN$CRASH.DATETIME <- as.numeric(collisions_test_kNN$CRASH.DATETIME)

#Setting the MIN and MIX of the training set
min_dt <- min(collisions_train_kNN$CRASH.DATETIME)
max_dt <- max(collisions_train_kNN$CRASH.DATETIME)

#Normalizing all sets with min/max
collisions_train_kNN$CRASH.DATETIME <- (collisions_train_kNN$CRASH.DATETIME - min_dt) / (max_dt - min_dt)
collisions_validation_kNN$CRASH.DATETIME <- (collisions_validation_kNN$CRASH.DATETIME - min_dt) / (max_dt - min_dt)
collisions_test_kNN$CRASH.DATETIME <- (collisions_test_kNN$CRASH.DATETIME - min_dt) / (max_dt - min_dt)

#Looking at the range of the datasets
range(collisions_train_kNN$CRASH.DATETIME)
```

```
## [1] 0 1
```

```
range(collisions_validation_kNN$CRASH.DATETIME)
```

```
## [1] 7.382537e-05 9.456196e-01
```

```
range(collisions_test_kNN$CRASH.DATETIME)
```

```
## [1] -1.217738e-05 9.067814e-01
```

Time has now been normalized across all sets

All columns have been normalized, except our target variable. When we created the dummy variables, the original variables were left behind, we can now delete those.

```
#Looking at the before
head(collisions_train_kNN)
```

```
## # A tibble: 6 × 21
##   BOROUGH CONTRIBUTING.FACTOR.VEHICLE.1 VEHICLE.TYPE.CODE.1 VEHICLE.TYPE.CODE.2
##   <chr>      <chr>                                <chr>          <chr>
## 1 BROOKLYN Unspecified                        Sedan          Other
## 2 BRONX     Unsafe Driving                          SUV            SUV
## 3 QUEENS    Unsafe Driving                          SUV            none
## 4 BROOKLYN Driving Conditions                Sedan          none
## 5 BRONX     Unsafe Driving                          SUV            none
## 6 QUEENS    Unspecified                        Sedan          none
## # i 17 more variables: CRASH.DATETIME <dbl>, SEVERITY.CLASS <chr>,
## #   BOROUGH_BRONX <int>, BOROUGH_BROOKLYN <int>, BOROUGH_MANHATTAN <int>,
## #   BOROUGH_QUEENS <int>, `BOROUGH_STATEN ISLAND` <int>,
## #   `CONTRIBUTING.FACTOR.VEHICLE.1_Driving Conditions` <int>,
## #   `CONTRIBUTING.FACTOR.VEHICLE.1_Unsafe Driving` <int>,
## #   CONTRIBUTING.FACTOR.VEHICLE.1_Unspecified <int>,
## #   VEHICLE.TYPE.CODE.1_Other <int>, VEHICLE.TYPE.CODE.1_Sedan <int>, ...
```

```
#removing the first 4 variables, the ones that were dummy coded
collisions_train_kNN <- collisions_train_kNN[ , -(1:4)]
collisions_validation_kNN <- collisions_validation_kNN[ , -(1:4)]
collisions_test_kNN <- collisions_test_kNN[ , -(1:4)]
```

```
#looking at the after
head(collisions_train_kNN)
```

```
## # A tibble: 6 × 17
##   CRASH.DATETIME SEVERITY.CLASS BOROUGH_BRONX BOROUGH_BROOKLYN BOROUGH_MANHATTAN
##   <dbl> <chr>                <int>          <int>          <int>
## 1    0.496 High                0              1              0
## 2    0.489 High                1              0              0
## 3    0.727 Low                 0              0              0
## 4    0.368 Moderate            0              1              0
## 5    0.171 Low                 1              0              0
## 6    0.391 High                0              0              0
## # i 12 more variables: BOROUGH_QUEENS <int>, `BOROUGH_STATEN ISLAND` <int>,
## #   `CONTRIBUTING.FACTOR.VEHICLE.1_Driving Conditions` <int>,
## #   `CONTRIBUTING.FACTOR.VEHICLE.1_Unsafe Driving` <int>,
## #   CONTRIBUTING.FACTOR.VEHICLE.1_Unspecified <int>,
## #   VEHICLE.TYPE.CODE.1_Other <int>, VEHICLE.TYPE.CODE.1_Sedan <int>,
## #   VEHICLE.TYPE.CODE.1_SUV <int>, VEHICLE.TYPE.CODE.2_none <int>,
## #   VEHICLE.TYPE.CODE.2_Other <int>, VEHICLE.TYPE.CODE.2_Sedan <int>, ...
```

For the target variable, similar to lab 1, it should not be normalized, but moved to its own object and factored for model prediction labels.

```
#training
train_labels_kNN <- factor(collisions_train_kNN[[2]]) #storing training labels
collisions_train_kNN <- collisions_train_kNN[ , -2] #removing from training set

#validation
validation_labels_kNN <- factor(collisions_validation_kNN[[2]]) #storing validation labels
collisions_validation_kNN <- collisions_validation_kNN[ , -2] #removing from validation set

#test
test_labels_kNN <- factor(collisions_test_kNN[[2]]) #storing test labels
collisions_test_kNN <- collisions_test_kNN[ , -2] #removing from test set

#confirming Levels
levels(train_labels_kNN)
```

```
## [1] "High"      "Low"       "Moderate"
```

Now our data is ready for modeling. For our first K value we will use the squareroot of the number of records and pick the closest odd number. The squareroot of 10775 records is 103. This is quite large, but we will see how it performs, and tune from there.

kNN Model 1

Installing the needed library

```
library(class)
```

Creating the model and predictions.

```
kNNpred1 <- knn(train = collisions_train_kNN, test = collisions_validation_kNN, cl = train_labels_kNN, k = 103)
```

To evaluate our model, we will be using the confusionMatrix() from the package caret. It will show stats such as accuracy, precision, and more. We will then use caret to further tune our model.

```
## package 'caret' successfully unpacked and MD5 sums checked
```

```
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
confusionMatrix(data = kNNpred1,
                 reference = validation_labels_kNN,
                 positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      473 281      288
##   Low       114 275      184
##   Moderate  172 218      304
##
## Overall Statistics
##
##           Accuracy : 0.4556
##           95% CI : (0.4351, 0.4762)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1844
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6232      0.3553      0.3918
## Specificity           0.6329      0.8059      0.7456
## Pos Pred Value        0.4539      0.4799      0.4380
## Neg Pred Value        0.7743      0.7126      0.7077
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.2049      0.1191      0.1317
## Detection Prevalence  0.4513      0.2482      0.3006
## Balanced Accuracy      0.6280      0.5806      0.5687
```

The model with a K of 103, does not appear to perform very well. With an accuracy of 46%, this is lower than chance. Sensitivity to correctly identify positive class "High" is a little better at 62%.

After trying a few K values we found that K=90 works the best for this method. Although still not much stronger.

```
#K=90
kNNpred2 <- knn(train = collisions_train_kNN, test = collisions_validation_kNN, cl = train_labels_kNN, k = 90)

confusionMatrix(data = kNNpred2,
                 reference = validation_labels_kNN,
                 positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      484 288      294
##   Low       111 268      181
##   Moderate  164 218      301
##
## Overall Statistics
##
##           Accuracy : 0.456
##           95% CI : (0.4356, 0.4766)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1851
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6377      0.3463      0.3879
## Specificity           0.6245      0.8098      0.7508
## Pos Pred Value        0.4540      0.4786      0.4407
## Neg Pred Value        0.7788      0.7107      0.7079
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.2096      0.1161      0.1304
## Detection Prevalence  0.4617      0.2425      0.2958
## Balanced Accuracy      0.6311      0.5780      0.5694
```

K=90 increases accuracy and sensitivity by about 1%

kNN Model 2

Now lets try the caret method from lab 8.

For this method, we have to add the target variables back to the datasets for the function to work.

```
#training
collisions_train_kNNc <- collisions_train_kNN #making a copy of the training set for caret
collisions_train_kNNc$SEVERITY.CLASS <- train_labels_kNN #adding labels back to train set

#validation
collisions_validation_kNNc <- collisions_validation_kNN #making a copy of the validation set
collisions_validation_kNNc$SEVERITY.CLASS <- validation_labels_kNN #adding labels back to validation
```

```
set.seed(300)
kNNm2 <- train(SEVERITY.CLASS~., data = collisions_train_kNNc, method = "knn")
kNNm2
```

```
## k-Nearest Neighbors
##
## 10775 samples
##    16 predictor
##    3 classes: 'High', 'Low', 'Moderate'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 10775, 10775, 10775, 10775, 10775, ...
## Resampling results across tuning parameters:
##
##  k  Accuracy  Kappa
##  5  0.3857902  0.07868689
##  7  0.3951475  0.09277056
##  9  0.4002413  0.10037857
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 9.
```

K=9 was selected as the best model, but the accuracy and kappa are very low. Seeing how it predicts.

```
set.seed(300)
kNNpred3 = predict(kNNm2, collisions_validation_kNNc)
confusionMatrix(data = kNNpred3,
                  reference = collisions_validation_kNNc$SEVERITY.CLASS,
                  positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      375 226      229
##   Low       180 296      253
##   Moderate  204 252      294
##
## Overall Statistics
##
##           Accuracy : 0.4179
##           95% CI : (0.3977, 0.4384)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2e-16
##
##           Kappa : 0.1271
##
##   McNemar's Test P-Value : 0.08366
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.4941      0.3824      0.3789
## Specificity           0.7065      0.7179      0.7025
## Pos Pred Value        0.4518      0.4060      0.3920
## Neg Pred Value        0.7404      0.6975      0.6908
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.1624      0.1282      0.1273
## Detection Prevalence  0.3595      0.3157      0.3248
## Balanced Accuracy      0.6003      0.5502      0.5407
```

As expected performance is very poor, this is our worst model yet.

Lets try tuning the model.

kNN Model 3

setting the control

```
ctrl = trainControl(method = "cv", number = 10, sampling = "up", selectionFunction = "oneSE")
```

setting the grid

```
grid = expand.grid(.k= c(9,50,75,90,103,110))
```

Adding weights to our positive class in the hopes of the model being able to predict “High” severity better.

```
weights = ifelse(collisions_train_kNNc$SEVERITY.CLASS == 'High', 3, 1)
```

Finding the best model

```
set.seed(300)
kNNm3 = train(SEVERITY.CLASS ~ .,
              data = collisions_train_kNNc,
              method = "knn",
              metric = "Accuracy",
              trControl = ctrl,
              weights = weights,
              tuneGrid = grid )
```

```
kNNm3
```

```
## k-Nearest Neighbors
##
## 10775 samples
##    16 predictor
##     3 classes: 'High', 'Low', 'Moderate'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 9698, 9696, 9697, 9698, 9697, 9698, ...
## Additional sampling using up-sampling
##
## Resampling results across tuning parameters:
##
##    k    Accuracy    Kappa
##    9  0.4142909  0.1213772
##   50  0.4453827  0.1677068
##   75  0.4474254  0.1707301
##   90  0.4504874  0.1752859
##  103  0.4507662  0.1756964
##  110  0.4515103  0.1768110
##
## Accuracy was used to select the optimal model using the one SE rule.
## The final value used for the model was k = 110.
```

Model still is not producing get stats. Predicting to see overall performance

```
set.seed(300)
kNNpred4 = predict(kNNm3, collisions_validation_kNNc)
confusionMatrix(data = kNNpred4,
                 reference = collisions_validation_kNNc$SEVERITY.CLASS,
                 positive = "High")
```



```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      475 287      286
##   Low       125 284      192
##   Moderate  159 203      298
##
## Overall Statistics
##
##           Accuracy : 0.4578
##           95% CI : (0.4373, 0.4784)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1877
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6258      0.3669      0.3840
## Specificity           0.6303      0.7935      0.7639
## Pos Pred Value        0.4532      0.4725      0.4515
## Neg Pred Value        0.7748      0.7131      0.7101
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.2057      0.1230      0.1291
## Detection Prevalence  0.4539      0.2603      0.2858
## Balanced Accuracy      0.6281      0.5802      0.5739
```

This model is a little better than last, but overall, still pretty terrible. Weights were unfortunately not very helpful.

Testing kNN

Best performing model was K=90, we will use it to test.

```
kNNpredtest <- knn(train = collisions_train_kNN, test = collisions_test_kNN, cl = train_labels_kNN, k = 90)

confusionMatrix(data = kNNpredtest,
                 reference = test_labels_kNN,
                 positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      455 293      296
##   Low       132 283      144
##   Moderate  171 198      337
##
## Overall Statistics
##
##           Accuracy : 0.4656
##           95% CI : (0.4451, 0.4862)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1994
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6003      0.3656      0.4337
## Specificity           0.6202      0.8202      0.7591
## Pos Pred Value        0.4358      0.5063      0.4773
## Neg Pred Value        0.7605      0.7194      0.7255
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.1971      0.1226      0.1460
## Detection Prevalence  0.4521      0.2421      0.3058
## Balanced Accuracy      0.6103      0.5929      0.5964
```

Our testing model performed a little worse than the training and validation, but not by much.

Boosting

As the kNN models are all underperforming at the same level, it appears to be an issue of underfitting. Our model might be too simple. To handle underfitting, we can use a more complex algorithm to try to better predict our data.

The regular `boosting()` function was attempted, but we could not get it to complete calibration. We try different variations and even let it run for 12 hours at one point and it would not finalize. So we decided to use caret's boosting model "gbm"

```
#copying training and validation to alter for gmb, leaving original intact
collisions_train_bo <- collisions_train
collisions_validation_bo <- collisions_validation

#factoring our target variable, needed for gbm
collisions_train_bo$SEVERITY.CLASS <- as.factor(collisions_train_bo$SEVERITY.CLASS)
collisions_validation_bo$SEVERITY.CLASS <- as.factor(collisions_validation_bo$SEVERITY.CLASS)

#creating the model, using methods from hw 8.
ctrl = trainControl(method = "cv", number = 10, selectionFunction = "oneSE")
grid = expand.grid(n.trees = 25, interaction.depth = 3, shrinkage = .1, n.minobsinnode = 10 )
set.seed(300)

modelbo = train(SEVERITY.CLASS ~ .,
                data = collisions_train_bo,
                method = "gbm",
                metric = "Accuracy",
                trControl = ctrl,
                tuneGrid = grid)
```

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.0986	nan	0.1000	0.0143
##	2	1.0893	nan	0.1000	0.0118
##	3	1.0822	nan	0.1000	0.0087
##	4	1.0763	nan	0.1000	0.0070
##	5	1.0716	nan	0.1000	0.0054
##	6	1.0677	nan	0.1000	0.0038
##	7	1.0650	nan	0.1000	0.0036
##	8	1.0623	nan	0.1000	0.0029
##	9	1.0603	nan	0.1000	0.0010
##	10	1.0591	nan	0.1000	0.0021
##	20	1.0491	nan	0.1000	-0.0004
##	25	1.0457	nan	0.1000	-0.0008

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.0986	nan	0.1000	0.0158
##	2	1.0885	nan	0.1000	0.0116
##	3	1.0814	nan	0.1000	0.0093
##	4	1.0752	nan	0.1000	0.0062
##	5	1.0708	nan	0.1000	0.0049
##	6	1.0667	nan	0.1000	0.0041
##	7	1.0639	nan	0.1000	0.0033
##	8	1.0615	nan	0.1000	0.0030
##	9	1.0592	nan	0.1000	0.0014
##	10	1.0578	nan	0.1000	0.0006
##	20	1.0481	nan	0.1000	0.0001
##	25	1.0447	nan	0.1000	0.0006

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.0986	nan	0.1000	0.0149
##	2	1.0893	nan	0.1000	0.0113
##	3	1.0818	nan	0.1000	0.0087
##	4	1.0765	nan	0.1000	0.0063
##	5	1.0721	nan	0.1000	0.0052
##	6	1.0686	nan	0.1000	0.0040
##	7	1.0658	nan	0.1000	0.0031
##	8	1.0636	nan	0.1000	0.0037
##	9	1.0612	nan	0.1000	0.0013
##	10	1.0599	nan	0.1000	0.0017
##	20	1.0494	nan	0.1000	-0.0006
##	25	1.0467	nan	0.1000	-0.0005

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.0986	nan	0.1000	0.0151
##	2	1.0893	nan	0.1000	0.0115
##	3	1.0815	nan	0.1000	0.0088
##	4	1.0759	nan	0.1000	0.0072
##	5	1.0714	nan	0.1000	0.0052
##	6	1.0676	nan	0.1000	0.0044
##	7	1.0647	nan	0.1000	0.0032
##	8	1.0622	nan	0.1000	0.0028
##	9	1.0602	nan	0.1000	0.0020

```

##      10      1.0586      nan      0.1000      0.0019
##      20      1.0479      nan      0.1000     -0.0003
##      25      1.0446      nan      0.1000     -0.0001
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0149
##      2      1.0883      nan      0.1000      0.0112
##      3      1.0809      nan      0.1000      0.0087
##      4      1.0751      nan      0.1000      0.0061
##      5      1.0711      nan      0.1000      0.0059
##      6      1.0675      nan      0.1000      0.0034
##      7      1.0643      nan      0.1000      0.0026
##      8      1.0623      nan      0.1000      0.0022
##      9      1.0603      nan      0.1000      0.0023
##     10      1.0586      nan      0.1000      0.0015
##     20      1.0483      nan      0.1000     -0.0005
##     25      1.0459      nan      0.1000      0.0001
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0152
##      2      1.0887      nan      0.1000      0.0115
##      3      1.0810      nan      0.1000      0.0087
##      4      1.0755      nan      0.1000      0.0065
##      5      1.0711      nan      0.1000      0.0052
##      6      1.0676      nan      0.1000      0.0043
##      7      1.0645      nan      0.1000      0.0032
##      8      1.0621      nan      0.1000      0.0021
##      9      1.0602      nan      0.1000      0.0022
##     10      1.0583      nan      0.1000      0.0011
##     20      1.0487      nan      0.1000      0.0004
##     25      1.0449      nan      0.1000      0.0001
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0157
##      2      1.0891      nan      0.1000      0.0114
##      3      1.0819      nan      0.1000      0.0092
##      4      1.0762      nan      0.1000      0.0068
##      5      1.0717      nan      0.1000      0.0052
##      6      1.0681      nan      0.1000      0.0057
##      7      1.0643      nan      0.1000      0.0035
##      8      1.0614      nan      0.1000      0.0017
##      9      1.0599      nan      0.1000      0.0029
##     10      1.0577      nan      0.1000      0.0007
##     20      1.0474      nan      0.1000     -0.0007
##     25      1.0448      nan      0.1000     -0.0000
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0151
##      2      1.0885      nan      0.1000      0.0116
##      3      1.0809      nan      0.1000      0.0092
##      4      1.0749      nan      0.1000      0.0063
##      5      1.0703      nan      0.1000      0.0050

```

```
##      6      1.0668      nan      0.1000      0.0039
##      7      1.0642      nan      0.1000      0.0019
##      8      1.0623      nan      0.1000      0.0030
##      9      1.0601      nan      0.1000      0.0021
##     10      1.0584      nan      0.1000      0.0015
##     20      1.0482      nan      0.1000     -0.0000
##     25      1.0452      nan      0.1000     -0.0003
##
```

```
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0149
##      2      1.0894      nan      0.1000      0.0114
##      3      1.0818      nan      0.1000      0.0088
##      4      1.0762      nan      0.1000      0.0057
##      5      1.0720      nan      0.1000      0.0045
##      6      1.0687      nan      0.1000      0.0041
##      7      1.0658      nan      0.1000      0.0033
##      8      1.0635      nan      0.1000      0.0029
##      9      1.0614      nan      0.1000      0.0010
##     10      1.0601      nan      0.1000     -0.0000
##     20      1.0507      nan      0.1000     -0.0006
##     25      1.0476      nan      0.1000      0.0001
##
```

```
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0145
##      2      1.0898      nan      0.1000      0.0118
##      3      1.0825      nan      0.1000      0.0094
##      4      1.0771      nan      0.1000      0.0065
##      5      1.0722      nan      0.1000      0.0053
##      6      1.0686      nan      0.1000      0.0044
##      7      1.0658      nan      0.1000      0.0035
##      8      1.0634      nan      0.1000      0.0022
##      9      1.0615      nan      0.1000      0.0026
##     10      1.0595      nan      0.1000      0.0010
##     20      1.0496      nan      0.1000      0.0001
##     25      1.0459      nan      0.1000     -0.0003
##
```

```
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.0986      nan      0.1000      0.0157
##      2      1.0891      nan      0.1000      0.0115
##      3      1.0822      nan      0.1000      0.0086
##      4      1.0762      nan      0.1000      0.0075
##      5      1.0717      nan      0.1000      0.0056
##      6      1.0680      nan      0.1000      0.0039
##      7      1.0651      nan      0.1000      0.0031
##      8      1.0628      nan      0.1000      0.0032
##      9      1.0603      nan      0.1000      0.0012
##     10      1.0589      nan      0.1000      0.0022
##     20      1.0492      nan      0.1000      0.0000
##     25      1.0464      nan      0.1000      0.0003
```

```
#predicting
predbo = predict(modelbo, collisions_validation_bo)
confusionMatrix(data = predbo,
                  reference = collisions_validation_bo$SEVERITY.CLASS,
                  positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      512 288      306
##   Low       93 260      168
##   Moderate 154 226      302
##
## Overall Statistics
##
##           Accuracy : 0.4651
##           95% CI : (0.4446, 0.4857)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1989
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6746      0.3359      0.3892
## Specificity           0.6168      0.8300      0.7521
## Pos Pred Value        0.4629      0.4990      0.4428
## Neg Pred Value        0.7947      0.7125      0.7087
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.2217      0.1126      0.1308
## Detection Prevalence  0.4790      0.2256      0.2954
## Balanced Accuracy      0.6457      0.5829      0.5706
```

The number of trees 25, 50, 60, and 100 were tested. 25 produced the best performance, though they are all still pretty consistent with the kNN models performances.

Boosting Model 2

Looking at the confusion matrix, it looks like the model is having trouble deciphering between the target classes. So we attempted a simpler target class of either Severe or non-Severe.

```

#binary switch
# Convert to binary classes, training
collisions_train_bo$SEVERITY.CLASS <- ifelse(
  collisions_train_bo$SEVERITY.CLASS == "High",
  "Severe",
  "Non-Severe"
)

#resetting factor
collisions_train_bo$SEVERITY.CLASS <- factor(
  collisions_train_bo$SEVERITY.CLASS,
  levels = c("Severe", "Non-Severe") # "Severe" comes first here
)

#validation
collisions_validation_bo$SEVERITY.CLASS <- ifelse(
  collisions_validation_bo$SEVERITY.CLASS == "High",
  "Severe",
  "Non-Severe"
)

#resetting factor
collisions_validation_bo$SEVERITY.CLASS <- factor(
  collisions_validation_bo$SEVERITY.CLASS,
  levels = c("Severe", "Non-Severe") # "Severe" comes first here
)

```

```
table(collisions_train_bo$SEVERITY.CLASS)
```

```
##
##      Severe Non-Severe
##      3614      7161
```

While there is some class imbalance with this method, the class imbalance is not too severe. We will add “smote” sampling to the model to help as well.

Trying the gbm model again to see if there are any improvements.

```

ctrl <- trainControl(method = "cv", number = 5, sampling = "smote")

set.seed(300)
modelbo2 = train(SEVERITY.CLASS ~ .,
  data = collisions_train_bo,
  method = "gbm",
  metric = "Accuracy",
  trControl = ctrl,
  tuneGrid = grid)

```

```
## Warning: package 'themis' was built under R version 4.3.3
```



```
## Loading required package: recipes
```

```
## Warning: package 'recipes' was built under R version 4.3.3
```

```
##  
## Attaching package: 'recipes'
```

```
## The following object is masked from 'package:stats':  
##  
##      step
```

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.3460	nan	0.1000	0.0202
##	2	1.3137	nan	0.1000	0.0164
##	3	1.2874	nan	0.1000	0.0133
##	4	1.2657	nan	0.1000	0.0110
##	5	1.2466	nan	0.1000	0.0094
##	6	1.2304	nan	0.1000	0.0080
##	7	1.2168	nan	0.1000	0.0065
##	8	1.2057	nan	0.1000	0.0055
##	9	1.1893	nan	0.1000	0.0077
##	10	1.1788	nan	0.1000	0.0052
##	20	1.0831	nan	0.1000	0.0038
##	25	1.0557	nan	0.1000	0.0011

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.3475	nan	0.1000	0.0196
##	2	1.3153	nan	0.1000	0.0157
##	3	1.2895	nan	0.1000	0.0132
##	4	1.2654	nan	0.1000	0.0119
##	5	1.2470	nan	0.1000	0.0097
##	6	1.2293	nan	0.1000	0.0090
##	7	1.2142	nan	0.1000	0.0077
##	8	1.2013	nan	0.1000	0.0062
##	9	1.1849	nan	0.1000	0.0077
##	10	1.1729	nan	0.1000	0.0058
##	20	1.0822	nan	0.1000	0.0028
##	25	1.0486	nan	0.1000	0.0023

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.3493	nan	0.1000	0.0178
##	2	1.3169	nan	0.1000	0.0166
##	3	1.2902	nan	0.1000	0.0133
##	4	1.2677	nan	0.1000	0.0108
##	5	1.2485	nan	0.1000	0.0096
##	6	1.2321	nan	0.1000	0.0082
##	7	1.2177	nan	0.1000	0.0068
##	8	1.2061	nan	0.1000	0.0060
##	9	1.1968	nan	0.1000	0.0046
##	10	1.1872	nan	0.1000	0.0047
##	20	1.0917	nan	0.1000	0.0023
##	25	1.0574	nan	0.1000	0.0028

##

##	Iter	TrainDeviance	ValidDeviance	StepSize	Improve
##	1	1.3473	nan	0.1000	0.0196
##	2	1.3159	nan	0.1000	0.0159
##	3	1.2900	nan	0.1000	0.0128
##	4	1.2683	nan	0.1000	0.0106
##	5	1.2492	nan	0.1000	0.0092
##	6	1.2335	nan	0.1000	0.0079
##	7	1.2207	nan	0.1000	0.0065
##	8	1.2100	nan	0.1000	0.0055
##	9	1.1963	nan	0.1000	0.0069

```
##      10      1.1843      nan      0.1000      0.0057
##      20      1.0886      nan      0.1000      0.0041
##      25      1.0571      nan      0.1000      0.0019
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.3472      nan      0.1000      0.0191
##      2      1.3155      nan      0.1000      0.0156
##      3      1.2892      nan      0.1000      0.0127
##      4      1.2672      nan      0.1000      0.0112
##      5      1.2494      nan      0.1000      0.0090
##      6      1.2301      nan      0.1000      0.0090
##      7      1.2158      nan      0.1000      0.0072
##      8      1.2035      nan      0.1000      0.0062
##      9      1.1915      nan      0.1000      0.0059
##     10      1.1815      nan      0.1000      0.0048
##     20      1.0839      nan      0.1000      0.0041
##     25      1.0531      nan      0.1000      0.0038
##
## Iter   TrainDeviance   ValidDeviance   StepSize   Improve
##      1      1.3465      nan      0.1000      0.0194
##      2      1.3144      nan      0.1000      0.0164
##      3      1.2876      nan      0.1000      0.0134
##      4      1.2657      nan      0.1000      0.0112
##      5      1.2469      nan      0.1000      0.0093
##      6      1.2314      nan      0.1000      0.0079
##      7      1.2145      nan      0.1000      0.0082
##      8      1.2026      nan      0.1000      0.0059
##      9      1.1924      nan      0.1000      0.0049
##     10      1.1795      nan      0.1000      0.0065
##     20      1.0868      nan      0.1000      0.0043
##     25      1.0545      nan      0.1000      0.0025
```

```
#predicting
```

```
predbo2 = predict(modelbo2, collisions_validation_bo)
confusionMatrix(data = predbo2,
                 reference = collisions_validation_bo$SEVERITY.CLASS,
                 positive = "Severe")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   Severe Non-Severe
##   Severe      277      341
##   Non-Severe  482     1209
##
##           Accuracy : 0.6436
##           95% CI : (0.6236, 0.6631)
##   No Information Rate : 0.6713
##   P-Value [Acc > NIR] : 0.9977
##
##           Kappa : 0.1522
##
##  Mcnemar's Test P-Value : 1.06e-06
##
##           Sensitivity : 0.3650
##           Specificity : 0.7800
##   Pos Pred Value : 0.4482
##   Neg Pred Value : 0.7150
##   Prevalence : 0.3287
##   Detection Rate : 0.1200
##   Detection Prevalence : 0.2676
##   Balanced Accuracy : 0.5725
##
##   'Positive' Class : Severe
##
```

While this did greatly improve our accuracy, it came at the cost of all other metrics. Note the no-information rate being .6713, and then accuracy is .6436, so this model does not offer any additional information further than labeling all observations as the majority class.

Testing best Boosting model

Out of the boosting models created, the first model performed the best. Lets test it on the testing data

```
collisions_test_bo <- collisions_test
collisions_test_bo$SEVERITY.CLASS <- as.factor(collisions_test_bo$SEVERITY.CLASS)

#predicting
predbo = predict(modelbo, collisions_test_bo)
confusionMatrix(data = predbo,
                 reference = collisions_test_bo$SEVERITY.CLASS,
                 positive = "High")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      493 304      315
##   Low       120 266      153
##   Moderate  145 204      309
##
## Overall Statistics
##
##           Accuracy : 0.4625
##           95% CI : (0.442, 0.4831)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1951
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6504      0.3437      0.3977
## Specificity           0.6009      0.8221      0.7722
## Pos Pred Value        0.4433      0.4935      0.4696
## Neg Pred Value        0.7786      0.7130      0.7165
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.2135      0.1152      0.1338
## Detection Prevalence  0.4816      0.2334      0.2850
## Balanced Accuracy      0.6256      0.5829      0.5849
```

All training, validation, and testing performed similiary. Testing model is mildly lower.

Decision Tree

Use objects: ***collisions_balanced, collisions_train, collisions_validation, and collisions_test***

We are building classification models to predict SEVERITY.CLASS (Low, Moderate, High) using our balanced dataset. We will use both the C5.0 and rpart decision tree algorithms and evaluate them on validation and test sets.

Convert Target to Factor and Ensure Consistent Levels

We need to convert SEVERITY.CLASS to a factor because R requires classification models to use factors for the target variable so they know it is a category. We also need to make sure the levels in SEVERITY.CLASS are the same in the training, validation, and test sets. This helps avoid errors and makes sure the model understands all class labels.

```
# Create Decision Tree-specific version of the balanced dataset
collisions_balanced_d_tree <- collisions_balanced

# Ensure SEVERITY.CLASS is a factor
collisions_balanced_d_tree$SEVERITY.CLASS <- factor(collisions_balanced_d_tree$SEVERITY.CLASS)

levels_all <- levels(collisions_balanced_d_tree$SEVERITY.CLASS)

collisions_train_d_tree <- collisions_train
collisions_validation_d_tree <- collisions_validation
collisions_test_d_tree <- collisions_test

# Apply consistent levels using the levels from the Decision Tree dataset
collisions_train_d_tree$SEVERITY.CLASS <- factor(collisions_train_d_tree$SEVERITY.CLASS, levels
= levels_all)
collisions_validation_d_tree$SEVERITY.CLASS <- factor(collisions_validation_d_tree$SEVERITY.CLAS
S, levels = levels_all)
collisions_test_d_tree$SEVERITY.CLASS <- factor(collisions_test_d_tree$SEVERITY.CLASS, levels =
levels_all)
```

Model 1: Decision Tree Using C5.0

Building a decision tree is using C50. This model was more recently created and is known to have better performance than rpart. However, it does not offer a data visualization function.

```
# Load libraries
library(caret)
library(C50)
```

```
## Warning: package 'C50' was built under R version 4.3.3
```

```
# Set seed for reproducibility
set.seed(300)
```

We use the C50 package and train() from caret to build a basic C5.0 decision tree.

```
# Train a default C5.0 decision tree
collisions_d_tree_c50 <- train(SEVERITY.CLASS ~ .,
                             data = collisions_train,
                             method = "C5.0")
```

Viewing our model through summary.

```
cat(head(capture.output(summary(collisions_d_tree_c50)), 30), sep = "\n")
```

```
##
## Call:
## (function (x, y, trials = 1, rules = FALSE, weights = NULL, control
## TRUE, noGlobalPruning = FALSE, CF = 0.25, minCases = 2, fuzzyThreshold
## = FALSE, sample = 0, earlyStopping = TRUE, label = "outcome", seed = 2718L))
##
##
## C5.0 [Release 2.07 GPL Edition]      Fri Apr 25 15:30:30 2025
## -----
##
## Class specified by attribute `outcome`
##
## Read 10775 cases (14 attributes) from undefined.data
##
## 5 attributes winnowed
## Estimated importance of remaining attributes:
##
##      6%  VEHICLE.TYPE.CODE.2Sedan
##      5%  VEHICLE.TYPE.CODE.2SUV
##      3%  VEHICLE.TYPE.CODE.1Sedan
##      3%  VEHICLE.TYPE.CODE.1SUV
##      3%  VEHICLE.TYPE.CODE.2Other
##      2%  CONTRIBUTING.FACTOR.VEHICLE.1Unsafe Driving
##      1%  BOROUGH.BROOKLYN
##      1%  BOROUGH.QUEENS
##
## ----- Trial 0: -----
##
## Rules:
```

Only showing first 30 lines to prevent a very long markdown.

This output shows that a C5.0 decision tree model was built to predict the variable outcome using 10,775 records and 14 features. During model training, five less useful features were automatically removed through a process called winnowing. The model used a single decision tree (not boosted), and no rule-based output was generated. It applied moderate pruning with a confidence factor of 0.25 and required at least two cases per leaf. Early stopping was enabled to avoid overfitting. The most influential variables in predicting the outcome included vehicle types such as sedans and SUVs, particularly those listed as the second vehicle, along with unsafe driving as a contributing factor and the boroughs Brooklyn and Queens.

Evaluate C5.0 on Validation Set

We predict on the validation set and evaluate with a confusion matrix.

```
# Predict
pred_c50_val <- predict(collisions_d_tree_c50, collisions_validation_d_tree)

# Match factor levels
pred_c50_val <- factor(pred_c50_val, levels = levels(collisions_validation_d_tree$SEVERITY.CLASS))

# Confusion matrix
conf_c50_val <- confusionMatrix(pred_c50_val, collisions_validation_d_tree$SEVERITY.CLASS)
print(conf_c50_val)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      549 350      349
##   Low        28 136        70
##   Moderate  182 288      357
##
## Overall Statistics
##
##           Accuracy : 0.4513
##           95% CI : (0.4308, 0.4718)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1785
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.7233      0.1757      0.4601
## Specificity           0.5490      0.9362      0.6934
## Pos Pred Value        0.4399      0.5812      0.4317
## Neg Pred Value        0.8021      0.6925      0.7173
## Prevalence            0.3287      0.3352      0.3361
## Detection Rate        0.2378      0.0589      0.1546
## Detection Prevalence  0.5405      0.1013      0.3582
## Balanced Accuracy      0.6362      0.5559      0.5767
```

The model correctly predicted about 45.1% of cases, which is better than random guessing (33.6%), and this improvement is statistically significant. The confusion matrix reveals that the model predicts “High” severity more accurately than “Low” or “Moderate.” Sensitivity is highest for “High” (72.3%) but very low for “Low” (17.6%), indicating poor detection of “Low” severity cases. The Kappa value of 0.1785 suggests only slight agreement beyond chance. Overall, the model performs moderately and could be improved with further tuning or different modeling approaches.

Final Evaluation of C5.0 on Test Set


```
# Predict on test set
pred_c50_test <- predict(collisions_d_tree_c50, collisions_test_d_tree)
pred_c50_test <- factor(pred_c50_test, levels = levels(collisions_test_d_tree$SEVERITY.CLASS))

# Confusion matrix
conf_c50_test <- confusionMatrix(pred_c50_test, collisions_test_d_tree$SEVERITY.CLASS)
print(conf_c50_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      539 394      353
##   Low        28 124       48
##   Moderate  191 256      376
##
## Overall Statistics
##
##           Accuracy : 0.45
##           95% CI : (0.4295, 0.4705)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1768
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.7111    0.16021    0.4839
## Specificity           0.5184    0.95049    0.7082
## Pos Pred Value        0.4191    0.62000    0.4569
## Neg Pred Value        0.7859    0.69180    0.7301
## Prevalence            0.3283    0.33521    0.3365
## Detection Rate        0.2334    0.05370    0.1628
## Detection Prevalence  0.5570    0.08662    0.3564
## Balanced Accuracy      0.6147    0.55535    0.5961
```

The model's performance on the test set is similar to the validation set, with an overall accuracy of 45%, which is better than random guessing (33.65%). The confusion matrix shows that the model predicts "High" severity cases fairly well but struggles with "Low" severity. Sensitivity is 71.1% for "High" but only 16.0% for "Low," indicating poor detection of "Low" severity. The Kappa value is 0.1768, suggesting slight agreement beyond chance. Overall, the model's performance is moderate, and there's potential for improvement, especially in detecting "Low" severity cases.

Model 2: Decision Tree Using rpart

The reason for using this library is because it provides visualization of our decision tree.

```
# Load library  
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.3.3
```

```
# Set seed for reproducibility  
set.seed(300)  
  
# Train the decision tree model  
# This builds a basic decision tree using the training set.  
collisions_d_tree_rpart <- rpart(SEVERITY.CLASS ~ .,  
                                data = collisions_train_d_tree,  
                                method = "class")
```

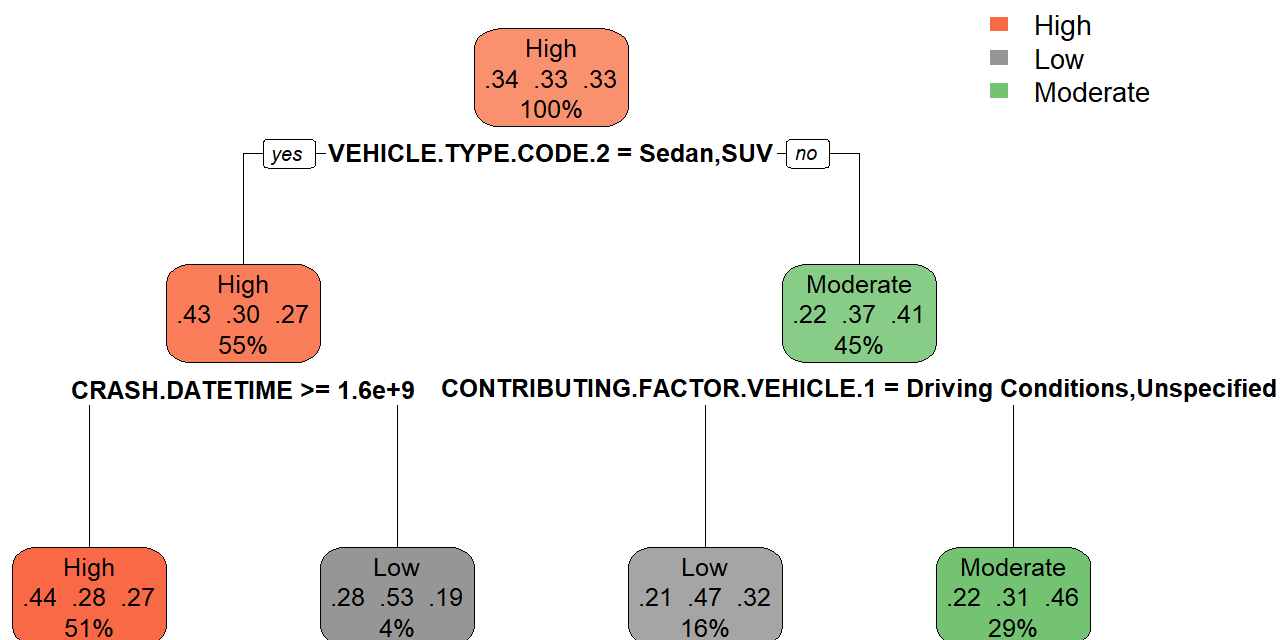
Visualize the Tree

This provides a visual overview of the decision tree structure and the most important splits.

```
# Load visualization package  
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.3.3
```

```
# Plot the rpart tree  
rpart.plot(collisions_d_tree_rpart)
```



The decision tree first splits based on whether the second vehicle involved is a Sedan or SUV. If yes, it checks if the crash occurred after a certain date; newer crashes tend to be classified as High severity. If not a Sedan or SUV, the model looks at the contributing factor. If the factor is related to Driving Conditions or is Unspecified, the crash is likely Moderate severity. Otherwise, crashes are more often classified as Low severity.

View Tree Summary

This shows more detailed information about the splits, nodes, and class probabilities.

```
# Display the first few lines of the tree summary
# Only showing first 30 lines to prevent a very long markdown.
cat(head(capture.output(summary(collisions_d_tree_rpart)), 30), sep = "\n")
```

```
## Call:
## rpart(formula = SEVERITY.CLASS ~ ., data = collisions_train_d_tree,
##       method = "class")
##       n= 10775
##
##           CP nsplit rel error   xerror   xstd
## 1 0.13098729    0 1.0000000 1.0205279 0.006771640
## 2 0.03519062    1 0.8690127 0.8690127 0.007160094
## 3 0.01605921    2 0.8338221 0.8352185 0.007203667
## 4 0.01000000    3 0.8177629 0.8194386 0.007218896
##
## Variable importance
##           VEHICLE.TYPE.CODE.2 CONTRIBUTING.FACTOR.VEHICLE.1
##                   61                                21
##           CRASH.DATETIME                                ZIP.CODE
##                   13                                3
##           BOROUGH
##                   3
##
## Node number 1: 10775 observations,   complexity param=0.1309873
##   predicted class=High   expected loss=0.664594   P(node)=1
##   class counts:  3614  3583  3578
##   probabilities: 0.335 0.333 0.332
##   left son=2 (5962 obs) right son=3 (4813 obs)
##   Primary splits:
##       VEHICLE.TYPE.CODE.2      splits as  RRLL, improve=188.85700, (0 missing)
##       CRASH.DATETIME          < 1585917000 to the left, improve= 44.36363, (0 missing)
##       CONTRIBUTING.FACTOR.VEHICLE.1 splits as  LLR, improve= 29.61234, (0 missing)
##       ZIP.CODE                < 10028.5   to the right, improve= 17.18069, (0 missing)
##       BOROUGH                 splits as  LLRLL, improve= 12.44315, (0 missing)
```

The decision tree was trained on 10,775 observations to classify SEVERITY.CLASS. The most important variable was VEHICLE.TYPE.CODE.2, followed by CONTRIBUTING.FACTOR.VEHICLE.1 and CRASH.DATETIME. The tree's initial split was based on vehicle type, and the predicted class at the root node was "High" with nearly even class distribution. The complexity parameter (CP) table shows that after three main splits, the relative error decreased from 1 to around 0.82. The model also identified surrogate splits, with CONTRIBUTING.FACTOR.VEHICLE.1 used as a backup when primary data is missing.

Evaluate rpart on Validation Set

This helps assess how well the model performs on unseen validation data.

```
# Predict on validation set
pred_rpart_val <- predict(collisions_d_tree_rpart,
                          newdata = collisions_validation_d_tree,
                          type = "class")

# Create confusion matrix
conf_rpart_val <- confusionMatrix(pred_rpart_val,
                                   collisions_validation_d_tree$SEVERITY.CLASS)

# Print results
print(conf_rpart_val)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      531 313      323
##   Low       95 212      173
##   Moderate 133 249      280
##
## Overall Statistics
##
##           Accuracy : 0.443
##           95% CI : (0.4227, 0.4636)
##   No Information Rate : 0.3361
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.166
##
##   Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6996      0.27390      0.3608
## Specificity           0.5897      0.82541      0.7508
## Pos Pred Value        0.4550      0.44167      0.4230
## Neg Pred Value        0.8004      0.69273      0.6988
## Prevalence            0.3287      0.33521      0.3361
## Detection Rate        0.2300      0.09181      0.1213
## Detection Prevalence  0.5054      0.20788      0.2867
## Balanced Accuracy      0.6446      0.54965      0.5558
```

The model achieved an accuracy of 44.3%, which is better than random guessing (No Information Rate = 33.6%). The kappa value is 0.166, indicating poor agreement beyond chance. Sensitivity (true positive rate) is highest for the High class at 70%, but its precision (Positive Predictive Value) is only 45.5%, meaning many predictions are incorrect. The Low and Moderate classes have lower sensitivity (27% and 36%), showing the model struggles to correctly identify those cases. Overall, the model favors predicting “High” and performs moderately on class separation.

Final Evaluation of rpart on Test Set

```
# Predict on test set
pred_rpart_test <- predict(collisions_d_tree_rpart,
                           newdata = collisions_test_d_tree,
                           type = "class")

# Create confusion matrix
conf_rpart_test <- confusionMatrix(pred_rpart_test,
                                   collisions_test_d_tree$SEVERITY.CLASS)

# Print results
print(conf_rpart_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      521 332      334
##   Low        85 228      130
##   Moderate  152 214      313
##
## Overall Statistics
##
##           Accuracy : 0.4599
##           95% CI : (0.4395, 0.4805)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1915
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6873    0.29457    0.4028
## Specificity           0.5706    0.85993    0.7611
## Pos Pred Value        0.4389    0.51467    0.4610
## Neg Pred Value        0.7888    0.70740    0.7153
## Prevalence            0.3283    0.33521    0.3365
## Detection Rate        0.2256    0.09874    0.1356
## Detection Prevalence  0.5141    0.19186    0.2941
## Balanced Accuracy     0.6290    0.57725    0.5820
```

The decision tree achieved 45.99% accuracy on the test set, slightly better than random guessing (No Information Rate = 33.65%). The Kappa value of 0.1915 shows low agreement beyond chance. The model performs best on the High class, with 69% sensitivity, but only 44% precision, meaning many predictions are incorrect. Sensitivity is lower for Low (29%) and Moderate (40%) classes, indicating difficulty identifying those cases. Overall, the model tends to overpredict “High” severity, and though performance is slightly better than validation, class separation remains weak.

The test set showed slightly better performance than the validation set, with accuracy increasing from 44.17% to 45.99% and kappa from 0.16 to 0.19. Sensitivity for High severity remained strong, while recall and precision for Low and Moderate classes improved. Overall, the model generalizes slightly better on unseen data but still favors predicting High severity.

Test Output Comparison: C5.0 versus rpart

Both models performed similarly overall, with rpart achieving a slightly higher accuracy of 45.99% compared to 45% for C5.0, and a higher kappa value (0.1915 vs. 0.1768), suggesting slightly better agreement beyond chance. C5.0 had higher sensitivity for the High and Moderate classes but lower for Low. In contrast, rpart showed better balance across classes, with improved sensitivity for the Low class and slightly higher precision for High and Moderate. Overall, rpart offered a more balanced classification, while C5.0 was slightly stronger at identifying High severity crashes.

So, if the goal is balanced performance across all classes, go with rpart. If the goal is about catching High severity crashes, then C5.0 might still be useful.

Model Improvements, Predictions, and Evaluation C5.0 and rpart

Improving rpart with smaller cp (less pruning).

This may improve accuracy or recall for minority classes by allowing more splits. Be cautious of overfitting.

```
# Train a more complex rpart tree
collisions_d_tree_rpart_cp <- rpart(SEVERITY.CLASS ~ .,
                                   data = collisions_train_d_tree,
                                   method = "class",
                                   control = rpart.control(cp = 0.0005))

# Predict on test set
pred_rpart_cp <- predict(collisions_d_tree_rpart_cp,
                        newdata = collisions_test_d_tree,
                        type = "class")

# Evaluate
conf_rpart_cp <- confusionMatrix(pred_rpart_cp, collisions_test_d_tree$SEVERITY.CLASS)
print(conf_rpart_cp)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      457 286      296
##   Low       135 298      184
##   Moderate  166 190      297
##
## Overall Statistics
##
##           Accuracy : 0.4556
##           95% CI : (0.4351, 0.4762)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1845
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6029      0.3850      0.3822
## Specificity           0.6248      0.7922      0.7676
## Pos Pred Value        0.4398      0.4830      0.4548
## Neg Pred Value        0.7630      0.7187      0.7101
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.1979      0.1291      0.1286
## Detection Prevalence  0.4500      0.2672      0.2828
## Balanced Accuracy      0.6138      0.5886      0.5749
```

The improved rpart model with a lower complexity parameter achieved 45.56% accuracy and a kappa of 0.1845, indicating modest agreement beyond chance. It performed best on the High severity class with a sensitivity of 60.3% and moderate precision. Performance on the Low and Moderate classes was lower but more balanced than before, with sensitivities around 38%. Balanced accuracy across all classes ranged from 57% to 61%, showing a slight improvement in handling class distribution.

Compared to the original rpart test model, the more complex version slightly reduced accuracy from 45.99% to 45.56% and kappa from 0.1915 to 0.1845. The original model had higher sensitivity for the High class at 68.7%, while the updated version dropped to 60.3%. However, the new model improved sensitivity for the Low class from 29.5% to 38.5%, offering better balance. Precision remained similar across both models. While the original tree was slightly more accurate overall, the updated version provided a more even performance across severity classes, particularly improving detection of Low severity crashes.

Improving C5.0 with Boosting (trials)

We can increase trials to apply boosting, helping the model learn from its previous mistakes.

Boosting often improves accuracy but may still favor the dominant class unless addressed.


```
# Load the C50 library
library(C50)

# Create predictor and target sets
x_train_d_tree <- collisions_train_d_tree[, names(collisions_train_d_tree) != "SEVERITY.CLASS"]
y_train_d_tree <- collisions_train_d_tree$SEVERITY.CLASS

# Ensure all character columns are factors (C5.0 requires this)
x_train_d_tree[] <- lapply(x_train_d_tree, function(x) if (is.character(x)) as.factor(x) else x)
collisions_test_d_tree[] <- lapply(collisions_test_d_tree, function(x) if (is.character(x)) as.factor(x) else x)

# Ensure target levels match
collisions_test_d_tree$SEVERITY.CLASS <- factor(collisions_test_d_tree$SEVERITY.CLASS, levels = levels(y_train_d_tree))
```

```
# Remove CRASH.DATETIME from predictors
x_train_d_tree <- x_train_d_tree[, names(x_train_d_tree) != "CRASH.DATETIME"]
collisions_test_d_tree <- collisions_test_d_tree[, names(collisions_test_d_tree) != "CRASH.DATETIME"]

# Train boosted model
collisions_d_tree_c50_boost <- C5.0(x = x_train_d_tree, y = y_train_d_tree, trials = 20)

# Predict on test set
pred_c50_boost <- predict(collisions_d_tree_c50_boost, collisions_test_d_tree)

# library(caret) # in case it's not already loaded

# Evaluate
conf_c50_boost <- confusionMatrix(pred_c50_boost, collisions_test_d_tree$SEVERITY.CLASS)
print(conf_c50_boost)
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      465 314      315
##   Low       117 240      180
##   Moderate  176 220      282
##
## Overall Statistics
##
##           Accuracy : 0.4275
##           95% CI : (0.4072, 0.4479)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1425
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6135      0.3101      0.3629
## Specificity           0.5945      0.8065      0.7415
## Pos Pred Value        0.4250      0.4469      0.4159
## Neg Pred Value        0.7588      0.6986      0.6965
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.2014      0.1039      0.1221
## Detection Prevalence  0.4738      0.2326      0.2936
## Balanced Accuracy      0.6040      0.5583      0.5522

```

The boosted C5.0 model with 20 trials achieved 42.75% accuracy and a kappa of 0.1425, which is lower than the original and improved rpart models. Sensitivity was highest for the High class at 61.4%, but lower for Moderate and Low at 36.3% and 31.0%, respectively. Precision ranged from 41% to 45%, with overall performance leaning toward High severity predictions. Balanced accuracy across classes was moderate, between 55% and 60%. While boosting helped retain good detection for High severity crashes, it did not outperform the rpart models and showed reduced balance and accuracy overall.

Improving C5.0 with Cost Matrix

To penalize certain misclassifications more heavily such as predicting High severity as Low, we define a cost matrix that assigns higher penalties to more serious errors. This shifts the model's focus toward reducing costly misclassifications, improving practical performance even if overall accuracy stays the same.

```
# Prepare predictor data (remove CRASH.DATETIME and ensure factors)
x_cost_d_tree <- collisions_train_d_tree[, names(collisions_train_d_tree) != "SEVERITY.CLASS"]
x_cost_d_tree <- x_cost_d_tree[, names(x_cost_d_tree) != "CRASH.DATETIME"]
x_cost_d_tree[] <- lapply(x_cost_d_tree, function(x) if (is.character(x)) as.factor(x) else x)

# Target variable
y_cost_d_tree <- collisions_train_d_tree$SEVERITY.CLASS

# Clean test set to match
x_test_cost <- collisions_test_d_tree[, names(collisions_test_d_tree) != "CRASH.DATETIME"]
x_test_cost[] <- lapply(x_test_cost, function(x) if (is.character(x)) as.factor(x) else x)
collisions_test_d_tree$SEVERITY.CLASS <- factor(collisions_test_d_tree$SEVERITY.CLASS, levels =
levels(y_cost_d_tree))

# Define cost matrix (already done above)
severity_levels <- levels(y_cost_d_tree)
cost_matrix <- matrix(c(
  0, 1, 2,
  1, 0, 2,
  2, 2, 0
), nrow = 3, byrow = TRUE)
dimnames(cost_matrix) <- list(predicted = severity_levels, actual = severity_levels)

# Train cost-sensitive C5.0 model
collisions_d_tree_c50_cost <- C5.0(x = x_cost_d_tree, y = y_cost_d_tree, costs = cost_matrix)

# Predict and evaluate
pred_c50_cost <- predict(collisions_d_tree_c50_cost, x_test_cost)
conf_c50_cost <- confusionMatrix(pred_c50_cost, collisions_test_d_tree$SEVERITY.CLASS)
print(conf_c50_cost)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      478 322      322
##   Low       156 298      238
##   Moderate  124 154      217
##
## Overall Statistics
##
##           Accuracy : 0.4301
##           95% CI : (0.4097, 0.4505)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1466
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.6306      0.3850      0.27928
## Specificity           0.5848      0.7433      0.81854
## Pos Pred Value        0.4260      0.4306      0.43838
## Neg Pred Value        0.7641      0.7056      0.69129
## Prevalence            0.3283      0.3352      0.33651
## Detection Rate        0.2070      0.1291      0.09398
## Detection Prevalence  0.4859      0.2997      0.21438
## Balanced Accuracy      0.6077      0.5642      0.54891
```

The model achieved 43.01% accuracy and a kappa of 0.1466, which is slightly better than the boosted C5.0 model but still lower than both rpart models. It performed best on the High class with 63.1% sensitivity, while Low class detection improved to 38.5%. However, performance on the Moderate class dropped, with sensitivity at just 27.9%. Precision was consistent across all classes, around 43%. Balanced accuracy ranged from 54.9% to 60.8%. Overall, the cost matrix helped improve fairness between High and Low classes but reduced detection of Moderate crashes.

The baseline rpart model achieved the highest accuracy at 45.99% with a kappa of 0.1915, showing strong sensitivity for High severity but weaker performance on Low and Moderate. After adjusting the complexity parameter, the improved rpart model showed slightly lower accuracy at 45.56% but improved balance, particularly for detecting Low severity crashes.

The baseline C5.0 model performed similarly with 45.00% accuracy and kappa of 0.1768, favoring the High class. After applying boosting with 20 trials, accuracy dropped to 42.75% and kappa to 0.1425, with more emphasis on High severity but weaker balance overall. The cost-sensitive C5.0 model further shifted focus toward reducing severe misclassifications. It achieved 43.01% accuracy and kappa of 0.1466, improving Low class detection but reducing performance on Moderate.

Overall, while no model significantly outperformed the others, the original rpart model remained the most accurate, and the cost-sensitive C5.0 model offered more balanced treatment of misclassifications aligned with real-world impact.

Random Forest

```
## package 'irr' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
## package 'ROCR' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

```
## package 'randomForest' successfully unpacked and MD5 sums checked
```

```
##
## The downloaded binary packages are in
## C:\Users\knigh\AppData\Local\Temp\Rtmp8K3pY1\downloaded_packages
```

Model 3 Random Forest

We assign RF label for Random Forest to the sample train, validate and test set so we don't alter the original samples when trying multiple models.

```
collisions_trainRF <- collisions_train
collisions_validationRF <- collisions_validation
collisions_testRF <- collisions_test
```

Setting target variable as factor for the model to classify results.

```
collisions_trainRF$SEVERITY.CLASS <- as.factor(collisions_trainRF$SEVERITY.CLASS)
collisions_validationRF$SEVERITY.CLASS <- as.factor(collisions_validationRF$SEVERITY.CLASS)
collisions_testRF$SEVERITY.CLASS <- as.factor(collisions_testRF$SEVERITY.CLASS)
```

Checking to make sure target variable is now a factor instead of a character.

```
str(collisions_trainRF)
```

```
## tibble [10,775 × 7] (S3: tbl_df/tbl/data.frame)
## $ BOROUGH                : chr [1:10775] "BROOKLYN" "BRONX" "QUEENS" "BROOKLYN" ...
## $ ZIP.CODE                : num [1:10775] 11236 10453 11377 11236 10453 ...
## $ CONTRIBUTING.FACTOR.VEHICLE.1: chr [1:10775] "Unspecified" "Unsafe Driving" "Unsafe Drivin
g" "Driving Conditions" ...
## $ VEHICLE.TYPE.CODE.1     : chr [1:10775] "Sedan" "SUV" "SUV" "Sedan" ...
## $ VEHICLE.TYPE.CODE.2     : chr [1:10775] "Other" "SUV" "none" "none" ...
## $ CRASH.DATETIME          : POSIXct[1:10775], format: "2022-06-23 14:40:00" "2022-06-11
22:05:00" ...
## $ SEVERITY.CLASS          : Factor w/ 3 levels "High","Low","Moderate": 1 1 2 3 2 1 1 3
1 2 ...
```

Now we are ready to run the model.

```
set.seed(300)
collisions_modelRF <- randomForest(SEVERITY.CLASS ~ ., data = collisions_trainRF)
collisions_modelRF      #displaying model
```

```
##
## Call:
## randomForest(formula = SEVERITY.CLASS ~ ., data = collisions_trainRF)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 2
##
##              OOB estimate of  error rate: 54.4%
## Confusion matrix:
##              High  Low Moderate class.error
## High          2188  592      834  0.3945766
## Low           1365 1212     1006  0.6617360
## Moderate     1294  771     1513  0.5771381
```

Test the performance of the model on the test dataset.

```
collisions_predRF <- predict(collisions_modelRF, collisions_testRF)
head(collisions_predRF)
```

```
##          1          2          3          4          5          6
##      High      High      High Moderate      High      High
## Levels: High Low Moderate
```

```
table(collisions_predRF, collisions_testRF$SEVERITY.CLASS)
```

```
##
## collisions_predRF High Low Moderate
##           High      454 281      297
##           Low       127 269      153
##           Moderate  177 224      327
```

```
results_RF <- confusionMatrix(data= collisions_predRF,
                             reference = collisions_testRF$SEVERITY.CLASS,
                             )
results_RF
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      454 281      297
##   Low       127 269      153
##   Moderate  177 224      327
##
## Overall Statistics
##
##           Accuracy : 0.4547
##           95% CI : (0.4343, 0.4753)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.1831
##
##   McNemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.5989      0.3475      0.4208
## Specificity           0.6273      0.8176      0.7383
## Pos Pred Value        0.4399      0.4900      0.4492
## Neg Pred Value        0.7619      0.7131      0.7154
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.1966      0.1165      0.1416
## Detection Prevalence  0.4469      0.2378      0.3153
## Balanced Accuracy      0.6131      0.5826      0.5796
```

Accuracy for this model 3 is at 45% and from this we can conclude that the model might not be learning the patterns very well or there is an issue with imbalanced data. Imbalanced data can be ruled out since we checked and addressed imbalances prior to running the model. We need the model to do better than chance or 50% so we will fine tune the model. Before we start fine tuning we let's look at other model metrics to gain more insight of the model performance.

```
confusionMatrix(collisions_predRF, collisions_testRF$SEVERITY.CLASS)$byClass
```

```
##          Sensitivity Specificity Pos Pred Value Neg Pred Value Precision
## Class: High      0.5989446   0.6273372      0.4399225      0.7619421 0.4399225
## Class: Low       0.3475452   0.8175896      0.4899818      0.7130682 0.4899818
## Class: Moderate  0.4208494   0.7382507      0.4491758      0.7153700 0.4491758
##          Recall      F1 Prevalence Detection Rate
## Class: High      0.5989446 0.5072626  0.3282806      0.1966219
## Class: Low       0.3475452 0.4066515  0.3352100      0.1165006
## Class: Moderate  0.4208494 0.4345515  0.3365093      0.1416197
##          Detection Prevalence Balanced Accuracy
## Class: High      0.4469467      0.6131409
## Class: Low       0.2377653      0.5825674
## Class: Moderate  0.3152880      0.5795500
```

The confusion matrix results further confirms why the accuracy is low. Its able to predict high severity but the precision is low indicating there are lots of false positives. The model struggles to find low severity and moderate severity performs slightly better than low but not as good as high.

Model 3.1 Random Forest: Try adding Weights

We are giving more importance to classes Low and Moderate to try to improve performance.

```
set.seed(300)
collisions_modelRF2 <- randomForest(SEVERITY.CLASS ~ ., data = collisions_trainRF, classwt = c(H
igh = 1, Moderate = 1.5, Low = 2))
collisions_modelRF2      #displaying model
```

```
##
## Call:
## randomForest(formula = SEVERITY.CLASS ~ ., data = collisions_trainRF,      classwt = c(High
= 1, Moderate = 1.5, Low = 2))
##          Type of random forest: classification
##          Number of trees: 500
## No. of variables tried at each split: 2
##
##          OOB estimate of  error rate: 58.91%
## Confusion matrix:
##          High  Low Moderate class.error
## High      753 2024      837  0.7916436
## Low       419 2362      802  0.3407759
## Moderate  405 1861     1312  0.6333147
```

Test the performance of the model on the test dataset.

```
collisions_predRF2 <- predict(collisions_modelRF2, collisions_testRF)
head(collisions_predRF2)
```

```
##          1          2          3          4          5          6
##      High      Low      High Moderate      High      Low
## Levels: High Low Moderate
```



```
confusionMatrix(collisions_predRF2, collisions_testRF$SEVERITY.CLASS)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low Moderate
##   High      148  79      102
##   Low       440 515      407
##   Moderate  170 180      268
##
## Overall Statistics
##
##           Accuracy : 0.4032
##           95% CI : (0.3831, 0.4235)
##   No Information Rate : 0.3365
##   P-Value [Acc > NIR] : 1.302e-11
##
##           Kappa : 0.1031
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: High Class: Low Class: Moderate
## Sensitivity           0.1953      0.6654      0.3449
## Specificity           0.8833      0.4482      0.7715
## Pos Pred Value        0.4498      0.3781      0.4337
## Neg Pred Value        0.6919      0.7265      0.6990
## Prevalence            0.3283      0.3352      0.3365
## Detection Rate        0.0641      0.2230      0.1161
## Detection Prevalence  0.1425      0.5899      0.2676
## Balanced Accuracy      0.5393      0.5568      0.5582
```

Unfortunately adding the weights did not improve accuracy. It actually decreased.

We will tune the model to try to get better results with the first Random Forest model. Lets first check the parameters.

```
modelLookup('rf')
```

```
##   model parameter           label forReg forClass probModel
## 1    rf          mtry #Randomly Selected Predictors   TRUE   TRUE   TRUE
```

We will change the mtry parameter by first setting the control

```
rf_ctrl = trainControl(method = "cv", number = 10, selectionFunction = "oneSE")
```

Creating the grid with an mtry of c(2,4,6,8)

```
grid = expand.grid(.mtry = c(2, 4, 6, 8))
grid
```

```
##   .mtry
## 1     2
## 2     4
## 3     6
## 4     8
```

Lets fine tune.

```
set.seed(300)

modelrf <- train(
  SEVERITY.CLASS ~ .,
  data = collisions_trainRF, # training data set
  method = "rf",            # random forest
  metric = "Kappa",         # performance metric to optimize
  trControl = rf_ctrl,      # cross-validation settings
  tuneGrid = grid           # grid of mtry values
)

modelrf
```

```
## Random Forest
##
## 10775 samples
##    6 predictor
##    3 classes: 'High', 'Low', 'Moderate'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 9698, 9696, 9697, 9698, 9697, 9698, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##   2     0.4571672  0.1851108
##   4     0.4643141  0.1959280
##   6     0.4428759  0.1639576
##   8     0.4153139  0.1228377
##
## Kappa was used to select the optimal model using the one SE rule.
## The final value used for the model was mtry = 4.
```

Model 4 is the best model.

Conclusion:

All models performed very similarly, this could be a sign of the patterns in the data. It might possibly be difficult to distinguish crash severity from the information given. We could also be experiencing underfitting. Although when using boosting we would expect to see an increase in performance, but the output remained relatively the same. Even when turning the target variable as binary, it still ended up less useful than assigning all crashes to non-severe, the majority class. This data may not be appropriate for predicting crash severity, though it does still provide useful insights such as when most collisions occur and where as shown in the exploratory analysis. If we were to suggest a model, the decision tree C.50 performed the best out of all, giving slightly better than chance with an accuracy of .45, sensitivity of .7111, and capturing 539 severe crashes correctly.